

Le code: une data pas comme les autres. Quelles sont les (bonnes) pratiques d'un dépôt de code FAIR ?

Mémoire de recherche réalisé par :

Laetitia REVERSY

Jéromine ROTH

Sous la direction de :

Patrick RUCH, Professeur ordinaire

Genève, le 16 janvier 2026

Sciences de l'information

Haute École de Gestion de Genève (HEG-GE)

Déclaration

Ce travail de recherche est réalisé dans le cadre du Master en Sciences de l'information de la Haute école de gestion de Genève.

Les étudiant-e-s attestent que le travail rendu est le fruit de leur réflexion personnelle, a été rédigé de manière autonome sans avoir utilisé des sources autres que celles citées dans la bibliographie et a été vérifié par un logiciel de détection de plagiat.

Les étudiant-e-s acceptent, le cas échéant, la clause de confidentialité.

L'utilisation des conclusions et recommandations formulées dans ce travail, sans préjuger de leur valeur, n'engage ni la responsabilité des étudiant-e-s, ni celle du/de la directeur-trice de recherche.

Fait à Genève, le 16 janvier 2026

Laetitia REVERSY

Jérôme ROTH

Remerciements

Nous remercions M. Julien Gobeill, chargé de cours à la filière Sciences de l'information, qui a bien voulu réaliser une extraction de la base de données PubMed à la demande du Prof. Patrick Ruch pour les besoins de notre étude. Ce travail nous a permis de constituer le corpus qui fait l'objet de notre exploration.

Nous remercions également le Prof. Patrick Ruch pour sa patience et son écoute tout au long de la rédaction de ce travail.

Résumé

Résumé

Cette recherche analyse de manière exploratoire les pratiques actuelles de gestion de logiciels liés à des articles scientifiques comme données de la recherche. Le corpus porte sur un échantillon d'articles tirés de PubMed Central. Elle évalue la qualité et l'exploitabilité des codes sous l'angle des principes FAIR applicables au code. L'étude repose sur une base de données structurée issue de l'analyse de 56 articles scientifiques et de leurs dépôts et hébergements, notamment Zenodo et GitHub. L'objectif est de proposer de bonnes pratiques opérationnelles favorisant un partage ouvert, durable et réutilisable du logiciel de recherche.

Après une revue de littérature décrivant l'évolution des Principes FAIR des données statiques aux données logicielles, un compte-rendu est fait des données de la recherche exploratoire. Une évaluation systématique de la qualité des codes du corpus est exposée sous la forme de critères tels que la licence, les plateformes choisies pour diffuser le code (dépôt, développement, distribution de packages, etc.) ainsi que les formats, métadonnées et identifiants pérennes.

Les résultats sont analysés pour découvrir comment et dans quelle mesure les Principes peuvent être mis en œuvre de manière pratique par les chercheurs.

Fort de cette analyse, des outils sont proposés : un référentiel FAIR pour l'évaluation de la qualité de la gestion de code et une checklist FAIR de dépôt de code pour guider les auteurs dans le développement de software.

Mots clefs

FAIR – FAIR for software – Research Software – Dépôt de code – Gestion des données de la recherche – Open science – Reproductibilité – Réutilisabilité

Disponibilité des données

Le présent rapport et ses données est disponible sur le repository GitHub, à l'adresse <https://github.com/TinouNavigue/BonnesPratiquesFAIR4S>.

Les releases sont automatiquement versionnées sur Zenodo. Le DOI conceptuel du projet est <https://zenodo.org/records/18274947>.

Table des matières

Déclaration	i
Remerciements	ii
Résumé	iii
Disponibilité des données	iv
Liste des tableaux	viii
Liste des figures	ix
1. Introduction	1
1.1 Contexte	1
1.2 Objectifs	1
2. Méthodologie	1
2.1 Glossaire	1
2.2 Données	3
2.2.1 Fichier « extensions »	3
2.2.2 Requêtes PubMed	4
2.3 Méthode	5
2.4 Périmètre de l'étude	7
2.4.1 Champ matériel	7
2.4.2 Champ temporel et linguistique	7
2.4.3 Langages	7
2.4.4 Formats	7
2.4.5 Exclusions	7
2.5 Livrables	8
2.5.1 Documentation annexée	8
2.5.2 Documentation déposée sur Zenodo et GitHub	8
2.5.3 Dépôt et hébergements	8
3. Revue de littérature	8
3.1 Principes FAIR	8
3.1.1 F - Findable (trouvabilité)	9
3.1.2 A - Accessible	9
3.1.3 I - Interopérable	10
3.1.4 R - Réutilisable	10
3.2 Adaptation aux données logicielles	10
3.2.1 Notion de software	10
3.2.2 F - Findable	12
3.2.3 A - Accessible	13
3.2.4 I - Interopérable	13
3.2.5 R - Réutilisable	14
3.3 Élargissement de la thématique	15
3.3.1 Obstacles	15

3.3.2	Caractère fonctionnel	16
3.3.3	Rôle des institutions	17
3.3.4	Importance de l'hébergement	17
3.3.5	Méthode de gestion	17
3.3.6	Gestion de projet	17
4.	Résultats de l'exploration.....	18
4.1	Méthode	18
4.2	Données des articles	18
4.2.1	Format	18
4.2.2	Provenance	20
4.2.3	Date de publication	21
4.3	Fréquentation du code	21
4.4	F - Findable	22
4.4.1	Mise à disposition	22
4.4.2	Lien vers le code	26
4.4.3	Qualité des métadonnées	27
4.5	A - Accessible	27
4.5.1	Authentification et Autorisation	28
4.5.2	Pérennité des métadonnées	29
4.6	I - Interoperable	29
4.6.1	Standards	29
4.6.2	Langages	29
4.7	R - Reusable	31
4.7.1	Licence des articles	31
4.7.2	Licences du code	31
4.7.3	Correspondance entre licences	32
4.7.4	Réutilisation effective	33
5.	Analyse critique des résultats	34
5.1	Articles	34
5.1.1	Format des articles	34
5.2	F - Findable	34
5.2.1	Pérennité du dépôt	34
5.2.2	Structuration des métadonnées	35
5.2.3	Qualité des métadonnées	37
5.2.4	Granularité de l'identification pérenne	37
5.3	A - Accessible	37
5.3.1	Protocole d'accès	37
5.4	I - Interoperable	38
5.4.1	Standards	38
5.4.2	Formats image	38
5.5	Réutilisabilité technique	39

5.5.1	Qualité de la documentation	39
5.5.2	Aspects techniques	40
5.5.3	Aspects éthiques	41
5.5.4	Règles de contribution	41
5.5.5	Structuration de la documentation	42
5.5.6	Destinataires de -R-	42
5.5.7	Aspects juridiques	43
6.	Bilan.....	44
6.1.1	Nature du FAIR	44
6.1.2	Le bon et le mauvais élève	45
6.1.3	Proposition d'outils	46
7.	Conclusion.....	46
	Bibliographie.....	47
Annexe 1 :	Référentiel FAIR pour la gestion de code de la recherche	49
Annexe 2 :	Checklist FAIR pour software	51
Annexe 3 :	Méthodologie détaillée.....	52

Liste des tableaux

<i>Tableau 1 : Formats d'articles proposés par les revues scientifiques.....</i>	<i>19</i>
--	-----------

Liste des figures

Figure 1 : Extensions des fichiers du code	4
Figure 2 : Principes FAIR	9
Figure 3 : FAIR Principles for Research Software (FAIR4S).....	12
Figure 4 : Provenance des institutions à l'origine de la recherche	20
Figure 5 : Date des articles	21
Figure 6 : Succès du code.....	22
Figure 7 : Choix de la mise à disposition du code par les auteurs	24
Figure 8 : Choix de mise à disposition de 2001 à 2025.....	25
Figure 9 : Focus sur la mise à disposition de 2016 à 2025	25
Figure 10 : Emplacement du lien vers le code.....	26
Figure 11 : Lien de retour vers l'article.....	27
Figure 12 : Protocole d'accès au code.....	28
Figure 13 : Protocoles d'accès restreint au code.....	28
Figure 14 : Langages/formats/extensions des fichiers des codes	30
Figure 15 : Licence des articles	31
Figure 16 : Licence déclarée par mise à disposition	32
Figure 17 : Exemple de page GitHub.....	36
Figure 18 : Exemple de page GitLab	36
Figure 19 : Exemple de mise à disposition de code au format image.....	39
Figure 20 : Exemple de ReadMe insuffisant, purement technique	41
Figure 21 : Badges des métadonnées sur GitHub.....	41
Figure 22 : Arborescence de la documentation sur GitHub	42
Figure 23 : Documentation pour machines v. humains	43

1. Introduction

1.1 Contexte

Ce travail s'intéresse à la gestion des données de la recherche lorsque celles-ci prennent la forme de données logicielles, ainsi qu'aux pratiques actuelles de dépôt de code source. Il a pour objectif de dresser un état des lieux des pratiques existantes et de proposer un ensemble de bonnes pratiques de dépôt de code conformes aux principes FAIR, sur la base d'un échantillon d'articles scientifiques publiés sur PubMed Central et comportant du code source. La méthodologie adoptée est celle d'une recherche exploratoire (Chapitre 0). Après un état des lieux de la littérature décrivant l'évolution des Principes FAIR appliqués au code depuis 2016 (Chapitre 3), nous présenterons les résultats de l'étude exploratoire sur le corpus (Chapitre 4). Ceux-ci feront l'objet d'une analyse à la lumière de travaux de référence (Chapitre 5), avant la formulation de recommandations pour un dépôt de code FAIR et l'ouverture de perspectives de recherche (Chapitre 6).

1.2 Objectifs

La question de la gestion du code est complexe car elle est transversale et se situe dans un domaine caractérisé par une évolution très rapide et un fonctionnement par versions. Il s'agit dès lors de relever les critères qui distinguent la gestion du code de celle des autres types de datasets. En effet, le code est construit comme un assemblage de briques (les modules), indépendantes et réutilisables dans d'autres codes. On peut les combiner pour élaborer des scripts plus complexes. Cette modularité demande une bonne gestion du code, non seulement pour sa pérennisation (risque de perte des données nécessaires à son fonctionnement), mais aussi pour assurer que les licences déclarées permettent sa réutilisation.

Si la gestion de datasets fait l'objet de nombreuses références en littérature depuis plus de 10 ans, celle du code est en développement. Les objectifs de ce travail sont les suivants :

1. proposer un référentiel pour analyser la qualité des codes archivés,
2. établir un référentiel des bonnes pratiques de dépôt de code au regard des normes FAIR.

2. Méthodologie

2.1 Glossaire

Les définitions ci-dessous s'appliquent aux termes employés dans ce travail.

Articles	Les 56 articles scientifiques faisant l'objet du corpus étudié ici. Un article peut faire l'objet de plusieurs Mises à disposition (voir ce mot).
Auteur	Personne responsable ou ensemble de personnes collectivement responsables d'une publication scientifique (typiquement un article).
Base de données	Base élaborée pour la récolte et l'analyse des 56 articles du corpus.

Code Les termes Code et Software, lorsqu'ils sont employés isolément, sont utilisés de manière interchangeable, sauf précision.

- **Code source**

Forme du logiciel lisible par l'humain, non compilée. Elle nécessite une installation dans une infrastructure pour être exécutée. Typiquement mise à disposition sur une forge sous forme d'un ensemble de fichiers logiciels et d'une documentation.

- **« Exécutable » / Code sous forme exécutable / Code binaire**

Forme lisible par les machines. Disponible sous forme d'un « exécutable », i.e. un fichier binaire qui fait tourner le logiciel sur un simple clic, par exemple une application sous forme d'un fichier JAR ou ZIP.

- **Exécutabilité et « Code exécutable »**

Termes ambigus qui peuvent référer tant à du Code sous forme exécutable qu'à du Code source dont on veut préciser que la finalité est d'être exécuté. Le terme Exécutabilité peut référer à l'aspect fonctionnel (code binaire qui s'exécute correctement lorsqu'on le lance ou code source qui fonctionne et s'exécute correctement lorsqu'on le compile et le fait tourner).

Données / Data Ce terme vise tous types de données confondus ayant servi à l'élaboration d'une publication scientifique. Il vise également les métadonnées, dans la mesure où celle-ci font l'objet de dépôt et de gestion au même titre que les données¹.

- **Données statiques**

Données non dynamiques, non modifiables après publication : données de comptage, données statistiques, images d'instruments de mesure, données bibliographiques, etc., par exemple sous formats CSV ou XML.

- **Données dynamiques**

Données susceptibles d'être modifiées au cours de leur cycle de vie, typiquement données logicielles

Research Software Logiciel produit au cours d'une recherche scientifique faisant l'objet d'une publication. Ceci inclut les dossiers, algorithmes, scripts, workflow, et code exécutable, etc. Ceci inclut également les bibliothèques, packages, standards et autres dépendances ayant été produits durant cette recherche ou adaptés à celle-ci dans le cadre de l'utilisation du software de recherche.

¹ À titre d'exemple, les données de la présente recherche ont pour source les métadonnées trouvées sur les plateformes de dépôt et de développement de code.

Dans ce travail, sauf mention contraire, les termes « **code** » et « **software** » sont employés au sens de « Research Software ».

Mise à disposition (ne pas confondre avec Publication). Elle reflète les variables de la Table `Mise à disposition` de la base de données et décrit la manière dont les auteurs d'un article ont rendu leur code disponible. Elle comprend toute forme de distribution du code sur une plateforme de développement, de dépôt ou de distribution de packages, sur un site dédié au software (commercial ou non), sur le site d'une institution académique et sur le cloud personnel d'un producteur (type Google) ; la simple insertion de code, soit dans le texte de l'article, soit dans ses annexes ou sur la page des suppléments de sa page PubMed ; la mention des coordonnées de la personne à contacter pour obtenir le code, l'annonce d'un code qui n'est pas fourni et les liens morts vers le code. À noter qu'un Article peut avoir plusieurs Mises à disposition (par exemple sur GitHub et sur Zenodo).

Plateformes

- **Plateforme de dépôt**

Dépôt de données fournissant un identifiant pérenne, typiquement Zenodo.

- **Plateforme de développement / Forge**

Plateforme de versionnage permettant l'hébergement, la gestion, le partage et le suivi collaboratif du code source. Typiquement GitHub, GitLab et BitBuckets.

- **Plateforme de distribution de packages**

Service en ligne permettant de publier, partager et installer des bibliothèques ou modules logiciels, typiquement CRAN et CPAN.

Publication (Ne pas confondre avec Mise à disposition) : diffusion d'un article dans une revue scientifique ou sur PubMed.

2.2 Données

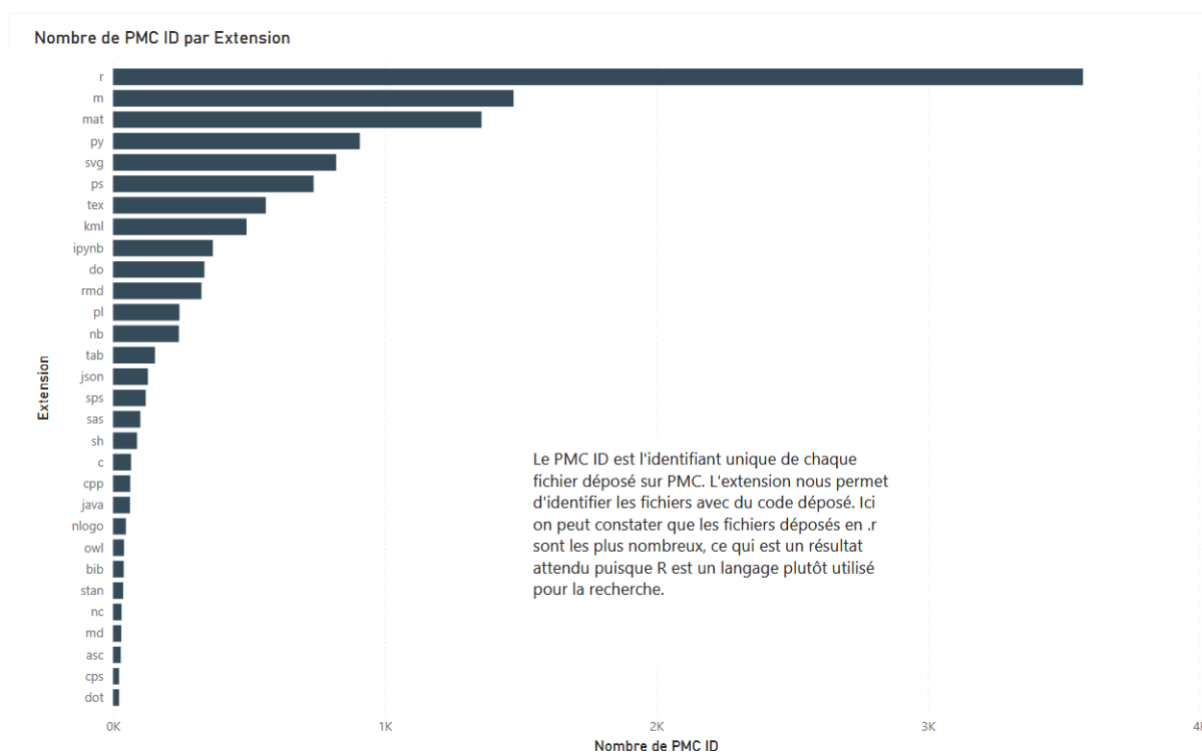
L'exploration se base sur deux sources, le fichier « extensions » et un corpus de requêtes effectuées dans PubMed.

2.2.1 Fichier « extensions »

Un fichier CSV de 13'000 fichiers a été extrait par M. Julien Gobeill à partir d'articles scientifiques sur PubMed Central. Il contient des fichiers de code, en langage de programmation exclusivement (par exemple PY), à l'exclusion des langages de structuration ou de représentation (XML, MD...). Chaque entrée contient i) le PMCID (identifiant de l'article sur PubMed) et ii) l'extension du fichier code y afférent.

Le rapport entre ces deux valeurs est illustré dans la visualisation suivante : On y constate la prépondérance des fichiers déposés en R, puis en M et MAT. Viennent ensuite les fichiers PY.

Figure 1 : Extensions des fichiers du code



(Extensions par PMID)

L'échelle a été conservée pour montrer la prépondérance des fichiers déposés en langage R. Une partie des résultats est absente de la figure, à savoir une centaine de formats qui comptent chacun entre 1 et 23 fichiers déposés. À partir de l'extension SAS, chaque format compte moins de 100 fichiers déposés.

2.2.2 Requêtes PubMed

Sur la base du fichier « extensions », onze extensions parmi les plus fréquentes dans le dataset ont été relevées par le Prof. Ruch de manière empirique :

R : 3570, M : 1475, MAT : 1357, PY : 909, SVG : 822, PS : 739, TEX : 563, KML : 492, IPYNB : 368, DO : 337, RMD : 326, PL : 245, NB : 243, TAB : 155, JSON : 129, SPS : 121, SAS : 101, SH : 89, C : 67, CPP : 64, JAVA : 63.

Une ou plusieurs requêtes correspondant aux onze extensions ont été formulées dans PMC PubMed Central (<https://pmc.ncbi.nlm.nih.gov/>) afin d'obtenir environ cinq articles par langage.

Au cours de ce processus, il s'est avéré que certaines extensions étaient difficiles à trouver au moyen de requêtes. D'autres étaient systématiquement associées et ont été groupées dans le même set de requêtes. Pour y remédier, des langages couplés tels que Cpp et C++ ont été groupés en un seul set de 5 requêtes. D'autres extensions ont donc été sélectionnées dans la liste des « extensions » jusqu'à obtenir au moins 5 articles liés à chaque extension. Lors de

ce choix, les langages non impératifs, qui n'avaient pas été exclus lors de la génération du fichier « extensions » ont été laissés de côté.

Au terme de la démarche, les extensions suivantes ont été sélectionnées :

- CPP : 5 articles
- PY : 7 articles
- MAT et MATLAB : 5 articles
- JAVA : 5 articles
- JSON : 2 articles
- R : 5 articles
- PERL : 5 articles
- SAS : 5 articles
- SHELL : 5 articles
- IPYNB : 5 articles
- SPSS : 4 articles
- NETLOGO : 4 articles

Les métadonnées des 56 articles renvoyés ont été intégrées dans une base de données créée à cet effet sur [Airtable.com](https://airtable.com). Celle-ci comprend les résultats des requêtes effectuées et notamment les données suivantes : les modalités de mise à disposition du code (langage, format, plateforme), la navigation entre l'article et le code, la présence d'identifiants pérennes, les licences du code et de l'article, la provenance des auteurs, le versionnage et des éléments sur la qualité de la documentation.

La démarche détaillée de constitution du corpus fait l'objet du fichier ReadMe annexé.

2.3 Méthode

Collecte

Le corpus d'échantillonnage a été analysé de manière exploratoire article par article pour identifier les usages courants de chercheurs. La collecte repose donc sur une inspection humaine et n'est pas automatisée.

Encodage

Les informations ont ensuite été encodées dans la base de données, soit sous forme de variables catégorielles, soit par codage qualitatif expert lorsque l'information nécessitait une interprétation contextuelle. À titre d'exemple, il existe des champs « Remarque sur les licences » et « Qualité du ReadMe ».

Les variables suivantes ont notamment fait l'objet d'une entrée dans la base de données :

Table `Articles` :

56 publications scientifiques comprenant du code associé.

Les variables suivantes sont extraites de chaque article :

- PMID (123456)
- Requête par lequel l'article a été trouvé (*"python" OR "py" AND code*)
- URL sur PubMed
- Titre
- DOI (de l'article)
- Domaine de la médecine concerné (vocabulaire contrôlé : *MeSH Terms*, complété par une terminologie ad hoc répondant au besoin de regroupement thématique plus large).
- Répartition par pays des institutions contributrices (ex. si l'auteur est affilié au Max Planck Institute of Mathematics in the Sciences, Leipzig : « Allemagne »)
- Date de publication
- Peer-reviewed (oui/non)
- Licence de l'article : présence, typologie et remarques
- Formats de dépôt de l'article
- Revue
- Remarque générale sur l'article

Table `Mises à disposition` :

Cette table décrit la manière dont les auteurs d'une publication ont rendu leur code disponible. Elle comprend les mises à disposition du code telles que définies dans le glossaire.

La table compte 81 mises à disposition de code source associé. La différence avec le nombre d'articles s'explique par le fait que la mise à disposition du code n'est pas systématique d'une part et que certains codes sont mis à disposition de multiples façons (par exemple GitHub et Zenodo).

Les variables suivantes sont extraites de chaque mise à disposition :

- Mode de mise à disposition
- DOI (du code)
- Date de première mise en ligne et remarques sur l'existence d'un historique ou de mises à jour
- Licence (type, typologies, trouvabilité, mention si celle-ci diffère de celle de l'article, remarques)
- Navigabilité entre plateformes, code et article
- Langage, format ou extension des fichiers du code source
- Présence d'exécutables
- Qualité de la documentation
- Remarques sur le versionnage
- Succès du code (compilation des variables de fréquentation du code : likes, forks, pull requests, download...)

À noter que certaines variables, telles que la provenance par pays, n'ont pas été exploitées car elles ont semblé peu pertinentes dès l'exploration. D'autres, comme l'exécutabilité, n'ont pas été approfondies du fait qu'elles dépassaient le cadre de ce travail.

Le fichier *Methodologie.md* décrit en détail la méthodologie adoptée dans cette recherche.

2.4 Périmètre de l'étude

2.4.1 Champ matériel

Les domaines sont ceux couverts par PubMed exclusivement et concernent donc la médecine et les sciences de la vie. Toutefois, la relève systématique de la variable « domaine » ne s'est pas avérée pertinente pour la recherche et n'a donc pas été exploitée. En effet, le standard de mots-clés de PubMed, les *MeSH Terms*, n'est pas utilisé de manière uniforme par les auteurs : les articles sont tantôt décrits en des termes très généraux (*Bioinformatic*, par exemple), tantôt très spécifiques (*pupillometry*). Ces derniers sont souvent appliqués à très peu d'articles du corpus (voire à un seul), dont la taille est modeste. Les *MeSH Terms* n'ont donc pas un grand poids statistique.

2.4.2 Champ temporel et linguistique

Les requêtes ont renvoyé des articles publiés entre le 1er février 2001 et le 11 septembre 2025, en anglais exclusivement. Ces paramètres n'ont pas été définis a priori.

2.4.3 Langages

D'autre part, la liste du fichier « extensions » a été filtrée par M. Gobeill pour exclure certaines extensions de code non impératif. Ont été ainsi exclues les extensions suivantes : TXT, CSV, HTML, XML, EPS, et HTM.

2.4.4 Formats

Certains auteurs joignent à leur article du code en format image, non copiable, a fortiori non exécutable. Or, l'exploration montre que l'on rencontre ce type de mise à disposition lorsque le code est non impératif (code de structuration ou de représentation par exemple). Dans la mesure où le fichier « extensions » exclut les formats de code non procédural, le nombre d'articles contenant ce format est sous-évalué dans l'échantillon. Il en résulte que ces pratiques d'insertion dans le texte, qui vont à l'encontre des principes FAIR, n'ont pas pu être évaluées précisément dans ce travail.

2.4.5 Exclusions

Caractère fonctionnel

Contrairement aux éléments formels, les aspects fonctionnels ne sont pas aisément quantifiables et dépassent donc le cadre des principes FAIR. Ainsi, tester l'aspect fonctionnel d'un code, c'est-à-dire sa capacité à produire des résultats corrects à chaque exécution peut nécessiter de hautes performances ou du calcul très long. Les mesures de sécurité et

d'efficacité computationnelle peuvent nécessiter un benchmarking scientifique systématique qui dépasse le contrôle des principes FAIR. (Lamprecht et al. 2020, p. 41)

De la même manière qu'un professionnel aide les chercheurs à évaluer la maturité FAIR de leurs données ne vérifiera ni leur qualité ni les processus de récolte, l'analyse FAIR d'un logiciel devrait se limiter aux aspects formels de la qualité du code, les aspects fonctionnels appartenant aux chercheurs et codeurs.

Plateformes

GitHub et Zenodo étant les seules plateformes apparaissant dans le corpus (à quelques exceptions près), les fonctionnalités des autres plateformes Git ou dépôts de données ne sont pas abordées.

2.5 Livrables

2.5.1 Documentation annexée

- Checklist FAIR for Software
- Référentiel FAIR for Software
- Methodologie.md

2.5.2 Documentation déposée sur Zenodo et GitHub

- Fichier « extensions »

2.5.3 Dépôt et hébergements

- Zenodo
- GitHub

3. Revue de littérature

3.1 Principes FAIR

Les principes FAIR de gestion de données scientifiques ont été formalisés lors de workshops, notamment en 2014 aux Pays-Bas, et ont fait l'objet d'une première publication par Wilkinson et al. en 2016.² Ils concernent la bonne gestion des données et leur dépôt en vue de leur réutilisation par la recherche scientifique. Ils visent à garantir la réutilisabilité des données scientifiques, laquelle est indispensable à la production du savoir par la communauté scientifique. Une bonne gestion des données après publication bénéficie notamment aux chercheurs, éditeurs, financeurs de la recherche et ingénieurs qui conçoivent des outils d'analyse des données. (Wilkinson et al. 2016, pp. 1-2)

² Jointly designing a data Fairport, Workshop tenu du 13 au 16 janvier 2014 à Leiden, Pays-Bas. Événement archivé à l'adresse : <https://www.lorentzcenter.nl/jointly-designing-a-data-fairport.html> [consulté le 27 décembre 2025] ; Wilkinson et al. 2016.

Dans cette perspective, les principes suivants ont été formalisés :

Figure 2 : Principes FAIR

Box 2 | The FAIR Guiding Principles

To be Findable:

- F1. (meta)data are assigned a globally unique and persistent identifier
- F2. data are described with rich metadata (defined by R1 below)
- F3. metadata clearly and explicitly include the identifier of the data it describes
- F4. (meta)data are registered or indexed in a searchable resource

To be Accessible:

- A1. (meta)data are retrievable by their identifier using a standardized communications protocol
 - A1.1 the protocol is open, free, and universally implementable
 - A1.2 the protocol allows for an authentication and authorization procedure, where necessary
- A2. metadata are accessible, even when the data are no longer available

To be Interoperable:

- I1. (meta)data use a formal, accessible, shared, and broadly applicable language for knowledge representation.
- I2. (meta)data use vocabularies that follow FAIR principles
- I3. (meta)data include qualified references to other (meta)data

To be Reusable:

- R1. meta(data) are richly described with a plurality of accurate and relevant attributes
 - R1.1. (meta)data are released with a clear and accessible data usage license
 - R1.2. (meta)data are associated with detailed provenance
 - R1.3. (meta)data meet domain-relevant community standards

(Wilkinson et al. 2016, p. 4)

Les Principes FAIR se déclinent comme suit³ :

3.1.1 F - Findable (trouvabilité)

Les données doivent être **découvrables/trouvables/localisables** par les humains comme par les machines. Il se peut en effet qu'un dataset soit présent sur le net mais qu'aucun lien n'y mène. Les données doivent donc être liées à un identifiant unique et pérenne (par exemple DOI) et associées à des métadonnées contextuelles lisibles et indexables par une machine, qui décrivent le contenu du dataset. Ainsi, une publication scientifique contiendra le lien pérenne vers le dataset ayant servi à son élaboration, et celui-ci sera accompagné de riches métadonnées descriptives permettant de le trouver, par exemple par mots-clefs. Enfin, le dataset devra être indexé sur un registre fouillable.

3.1.2 A - Accessible

Il ne suffit pas de trouver et identifier les données, encore doivent-elles être accessibles aux humains comme aux machines. En d'autres termes, les fichiers découverts doivent être **récupérables**. La récupération doit être possible par le biais d'un protocole standard de

³ Wilkinson et al. 2016, pp. 1-2 ; 2017, p. 4.

communication (par exemple HTTP) comprenant la possibilité d'y adjoindre une procédure d'autorisation ou d'authentification.

3.1.3 I - Interopérable

Le Principe -I- exige que la compatibilité entre machines : les données du code doivent être combinables avec celles d'autres logiciels. Ceci exige l'emploi de vocabulaires contrôlés, d'ontologies et formats standards.

Pour être interprétables et exploitables (-I- et -R-) par des machines, les métadonnées décrivant le dataset doivent respecter les standards applicables à la branche : normes de nommage, vocabulaires contrôlés, ontologies et formats standards.

À noter qu'en matière de logiciel, les Principes -I- et -R- se recoupent et sont réalisés par les mêmes critères (Chue Hong 2022, p. 10). Nous analyserons ces critères dans les parties **Reusable** du présent chapitre 3.2.5) et des chapitres **Résultats** (4.7) et **Analyse** 5.5).

3.1.4 R - Réutilisable

La réutilisation de données implique le droit et la possibilité technique, pour un chercheur intervenant après la publication des données, de i) **reproduire** la recherche (ses expériences), ii) l'**intégrer** dans une nouvelle recherche et iii) la **publier**.

Du point de vue juridique et éthique, les métadonnées doivent contenir des références claires aux auteurs de la publication et la déclaration d'une licence mentionnant clairement les droits de réutilisation et publication.

Du point de vue technique, les « riches métadonnées » contextuelles et descriptives mentionnées sous le Principe -A- doivent comprendre une pluralité d'attributs précis et pertinents qui correspondent aux standards du domaine étudié. Les données pourront ainsi être liées à d'autres sources de données et intégrées à des recherches subséquentes.

3.2 Adaptation aux données logicielles

3.2.1 Notion de logiciel

Les principes FAIR ont principalement été pensés dans le contexte de la gestion de données statiques. Rapidement, il s'est avéré nécessaire de préciser leur application aux données dynamiques telles que les logiciels, qui sont aussi des données de la recherche lorsqu'ils ont contribué à élaborer une publication scientifique. La notion de Research software est précisée comme suit :

« Research Software includes source code files, algorithms, scripts, computational workflows and executables that were created during the research process or for a research purpose. Software components (e.g., operating systems, libraries, dependencies, packages, scripts, etc.) that are used for research but were not created during or with a clear research intent should be considered software in research and not

Research Software. This differentiation may vary between disciplines. » (Grünpeter et al. 2021, p. 16)⁴

Ces questions sont déjà soulevées chez Wilkinson et al. 2017, où les auteurs précisent que l'application des principes FAIR couvre tout type de données, y.c. les workflows et leurs combinaisons/packages. Toutefois, l'adaptation pratique aux logiciels principe par principe a été établie par le « FAIR for Research Software Working Group » actif au sein de Research Data Alliance, dans Lamprecht et al. 2020.⁵

Ceux-ci rappellent ce qui distingue les softwares des data ordinaires :

Le processus de création : Les data (statiques) sont des observations ou faits probants, alors que les logiciels sont le résultat de processus créatifs qui fournissent un outil pour mener une action spécifique, par exemple avec les data. La finalité du software est d'être exécuté, pas celle de la donnée. (trad. libre de Lamprecht 2020, p. 39).

La dépendance à d'autres softwares : Il est rare qu'un software soit conçu de rien par un codeur. Les softwares sont composés de modules, leurs dépendances. Ainsi, un workflow orchestre plusieurs scripts pour exécuter un processus tel qu'un pipeline de données, une simulation, etc.

La durée de vie : Le software est voué à devenir obsolète dès que les dépendances évoluent et doit donc être versionné : chaque fixation de bug, adaptation aux évolutions des dépendances ou évolution du software suscite une évolution du code. L'historique de ces modifications est le versioning. La durée de vie du software est donc plus courte que celle des données qu'il orchestre.

⁴Cette définition exclut les dépendances du code produit dans le cadre de la recherche. Elle admet toutefois l'inclusion selon les disciplines, ce qui correspond à la définition du glossaire ci-dessus.

⁵ www.rd-alliance.org.

Partant de ces constats, Lamprecht et al. 2020 a adapté les Principes FAIR aux logiciels. Ils ont été consolidés par Chue Hong et al. (2022) comme suit :

Figure 3 : FAIR Principles for Research Software (FAIR4S)

The FAIR4RS Principles are:

F: Software, and its associated metadata, is easy for both humans and machines to find.
F1. Software is assigned a globally unique and persistent identifier. • F1.1. Components of the software representing levels of granularity are assigned distinct identifiers. • F1.2. Different versions of the software are assigned distinct identifiers. F2. Software is described with rich metadata. F3. Metadata clearly and explicitly include the identifier of the software they describe. F4. Metadata are FAIR, searchable and indexable.
A: Software, and its metadata, is retrievable via standardized protocols.
A1. Software is retrievable by its identifier using a standardized communications protocol. • A1.1. The protocol is open, free, and universally implementable. • A1.2. The protocol allows for an authentication and authorization procedure, where necessary. A2. Metadata are accessible, even when the software is no longer available.
I: Software interoperates with other software by exchanging data and/or metadata, and/or through interaction via application programming interfaces (APIs), described through standards.
I1. Software reads, writes and exchanges data in a way that meets domain-relevant community standards. I2. Software includes qualified references to other objects.
R: Software is both usable (can be executed) and reusable (can be understood, modified, built upon, or incorporated into other software).
R1. Software is described with a plurality of accurate and relevant attributes. • R1.1. Software is given a clear and accessible license. • R1.2. Software is associated with detailed provenance. R2. Software includes qualified references to other software. R3. Software meets domain-relevant community standards.

(Chue Hong et al. 2022, p. 5)

3.2.2 F - Findable

Le code évoluant par version, son identification pérenne ne suffit pas. Chaque version doit avoir un identifiant pérenne. En pratique, on ne saurait exiger aux multiples contributeurs d'un code de déposer et créer un identifiant pérenne pour chaque version. Puisque dans les faits, une très grande part des codeurs hébergent leur code sur la Plateforme de développement, GitHub, on doit examiner ce cas. GitHub crée bien un *hash* automatiquement à chaque *commit* mais GitHub étant une plateforme propriétaire (elle appartient à Microsoft depuis 2018), la pérennité de ses *hashs* ne peut être garantie. En effet, la nature et les missions des hébergeurs conditionnent l'application des pratiques FAIR. Une solution consiste à utiliser

l'API de GitHub qui dépose le code automatiquement à chaque release sur Zenodo avec un identifiant distinct des autres versions. (Lamprecht et al. 2020, p. 43)

3.2.3 A - Accessible

En matière de software, l'accessibilité devient la possibilité de récupérer le code (*retrieve*). Ceci nécessite l'emploi d'un protocole standardisé (typiquement HTTPS, accès via une page web). Le protocole doit être ouvert, gratuit et universellement implémentable, c'est-à-dire sans frais ni restriction technique. Tout comme pour les données statiques, le protocole doit permettre une procédure d'authentification ou d'autorisation. Les métadonnées descriptives doivent être disponibles encore après la fin du cycle de vie du software. (Chue Hong et al. 2022, pp. 8-9)

3.2.4 I - Interopérable

Le software doit interopérer avec d'autres softwares en écrivant et échangeant des données et métadonnées notamment via des APIs (*application programming interfaces*), en respectant i) les **standards de communication** de leur domaine scientifique. Il doit contenir des ii) **références** de qualité **aux objets externes** au code et indispensables à son fonctionnement.

Un petit rappel des notions s'impose : le code est construit sur la base des dépendances. Celles-ci sont

- un système d'exploitation en fond
 - sur lequel s'empilent les bibliothèques (Pandas)
 - lesquelles sont composées de modules (NumPy)
 - les scripts
 - les fichiers de configuration (paramètres, par exemple `config.yaml`)
 - les données que traite le logiciel (CSV ou JSON par exemple)
- Par ailleurs, des outils externes au code sont nécessaires pour son exécution, notamment
 - Compilateurs (GCC (GNU))
 - Interpréteurs (par exemple Python, R)
 - Bases de données (MySQL)
 - outils de build (CMake)

Les outils externes n'étant pas compris dans le code, les Principes -I- et -R- exigent qu'ils soient décrits en détail dans la documentation. Une alternative consiste encapsuler l'ensemble de l'environnement d'exécution dans une image Docker.

La qualité du code sert -I- et -R- (interopérabilité et réutilisabilité). Un software ne peut être réutilisé que s'il est pérenne et évolue de concert avec l'ensemble de l'écosystème auquel il appartient. Il s'agit de la maintenabilité (*sustainability*) du code source : il doit être modulaire, compréhensible, modifiable, analysable et doit pouvoir être testé (Lamprecht et al. 2020 p. 40). La maintenabilité implique le respect des normes de codage spécifiques à un langage ou à une communauté.

Un logiciel est interopérable lorsqu'il échange ses données dans des formats compatibles avec d'autres logiciels sans adaptation spécifique, par exemple en CSV ou JSON, dans des formats spécifiques à la discipline ou des API. Le partage doit se faire selon les conventions standard de sa communauté. À titre d'exemple, le FAIRSharing Registry (fairsharing.org/standards) recense des standards de gestion de données de la recherche.

3.2.5 R - Réutilisable

La faculté de réutiliser un software consiste en deux volets : légale et technique.

La réutilisation concerne le droit et la possibilité technique, pour un chercheur intervenant après la publication du code, de i) le **tester** pour **reproduire** la recherche, ii) le **remployer**, iii) l'**intégrer** dans une recherche iv) et le **publier**.

Du point de vue juridique, les métadonnées doivent contenir, d'une part, la déclaration d'une licence claire mentionnant les droits de réutilisation et publication et, d'autre part, la mention de la chaîne des contributeurs du dataset, selon le principe de provenance, afin de permettre de le citer de manière adéquate.

Du point de vue technique, les « riches métadonnées » contextuelles et descriptives exigées par le Principe -A- doivent comprendre une pluralité d'attributs précis et pertinents qui correspondent aux standards du domaine étudié. Les données pourront ainsi être liées à d'autres sources de données et intégrées à des recherches subséquentes.

Réutilisabilité légale

Pour remployer un code ou y faire référence, l'utilisateur doit être assuré de son droit de le faire. Le Principe -R- exige donc que l'auteur du code déclare une licence d'utilisation qui mentionne les droits relatifs à l'utilisation de tout ou partie du code, à son attribution (obligation de citation), sa modification et sa redistribution.

En pratique, cette licence peut être indiquée dans le ReadMe ou dans un fichier adhoc `Licence`, par exemple.

La licence est rarement rédigée pour la cause mais fait généralement référence à un modèle de type tel que les licences GNU (General Public Licences proposées par the Free Software Foundation)⁶ ou les modèles de Berkeley Software Distribution (BSD)⁷.

Aux fins de citation, les métadonnées du software doivent contenir les indications de provenance, à savoir les auteurs du code, le contexte de création (projet, but et cas d'usages, lien vers l'article scientifique), filiation (identifiant du dépôt d'origine, version). Les données de filiation du code sont aisément consultables dans les forges, du fait de l'enregistrement automatique des *releases*, des fusions et demandes de fusions et des *forks*⁸. Elles ne sont toutefois pas pérennisées, à moins d'être gelées sur un dépôt ou moissonnées par Software Heritage.

⁶ <https://www.gnu.org/licenses/licenses.en.html>

⁷ <https://opensource.org/license/bsd-3-clause>. De manière générale, les modèles de licences pour software sont répertoriées sur le site de l'Open Source Initiative (<https://opensource.org/license>).

⁸ Une *fork* consiste à copier le code pour en créer un nouveau qui évoluera de son côté, tout en conservant la trace de l'historique et informations de provenance de l'ancien code.

Finalement, les métadonnées de provenance et de versionnage doivent respecter les standards de la communauté, permettant aux utilisateurs subséquents de disposer de suffisamment d'information pour remployer, citer et diffuser le code.

Réutilisabilité technique

Le software doit être utilisable au moment de l'accès, mais également plus tard. Dans ce but, les utilisateurs doivent pouvoir l'utiliser dans d'autres softwares, mais également élaborer de nouveaux softwares sur sa base. Dans ce but, les métadonnées doivent donner la référence des dépendances du code.

Comme dit plus haut, les métadonnées doivent encore contenir des références qualifiées aux objets digitaux (données statiques ou softwares) qui sont nécessaires au fonctionnement du code. Ceci inclut les bibliothèques, les packages, l'environnement d'exécution, les outils pour compiler, les informations pour configurer le code, les API et les standards utilisés. Elles doivent également renvoyer aux objets matériels nécessaires à leur fonctionnement (instruments de mesure, échantillons, etc.). (Chue Hong et al. 2022, pp. 10-11)

Le Principe -R- requiert que le code soit compréhensible pour les humains comme pour les machines. Cependant, dans les faits, ces deux exigences peuvent s'opposer :

Comme dit plus haut, le software est un outil et il a pour finalité d'être exécuté. Il peut être fourni sous deux formes, qui satisfont toutes deux une partie du Principe -R- :

Code source : La forme du logiciel qui donne la meilleure connaissance d'un code à un utilisateur humain est le code source. C'est sous cette forme qu'il est disponible sur des forges telles que GitHub.

Cette forme n'est pas exécutable dès l'accès au code. Elle nécessite une connaissance informatique pour le compiler (l'installer dans une infrastructure, avec ses dépendances) et le faire tourner.

Ainsi, le code source est modifiable (selon le principe R) mais pas exécutable.

Exécutable ou code binaire : Au contraire, le code dit exécutable ou binaire est un code déjà compilé, dans un langage lisible par la machine. Un exécutable se présente sous la forme d'un fichier (JAR ou ZIP, par exemple), qui fait tourner le logiciel d'un clic. Cette forme est adaptée aux utilisateurs non codeurs et permet une utilisation immédiate du logiciel. Elle n'est en revanche pas lisible pour les codeurs humains.

Le code binaire est donc exécutable (selon le principe R) mais pas modifiable.

3.3 Élargissement de la thématique

3.3.1 Obstacles

La question des incitateurs au respect des principes FAIR est abordée en détail par Gomes et al. 2022. Ceux-ci font la liste des obstacles à la bonne gestion des données logicielles : les chercheurs ignorent souvent comment procéder au nettoyage et à la présentation de leurs données, surtout en présence de datasets importants et lorsque les workflows sont complexes. Ils craignent que l'on ne relève des erreurs dans la compilation.

Pour chacune de ces problématiques, les auteurs soulignent le rôle des institutions qui financent la recherche : celles-ci exigent de plus en plus un Data management plan et un dépôt de données précoces. Les comités de révision ont également une responsabilité et devraient acquérir une expertise en matière de datasets et de données logicielles des articles. Les institutions qui ne l'ont pas encore fait devraient investir dans des stockages sécurisés pour les datasets de leurs auteurs et ceux-ci devraient être formés au dépôt sur des plateformes comme Zenodo. Le manque d'expertise des auteurs est acceptable et il vaut mieux un partage peu FAIR que pas de partage du tout, sans compter que les données peuvent être versionnées si un utilisateur remarque une incohérence après leur dépôt.

Sensibiliser les auteurs permet de tempérer leurs craintes. On peut ainsi rappeler que les datasets pauvres et de mauvaise qualité apparente concernant des données non encore diffusées renferment potentiellement une grande valeur. S'agissant de la peur d'être doublé à la publication (*scooping*), on peut rappeler qu'il existe des serveurs de prépublication (*pre-print*) que seule la communauté peut consulter aux fins de conseil et d'amélioration. En cas d'utilisation abusive des données par d'autres auteurs, ces serveurs permettent de revendiquer la paternité d'un projet de recherche.

En moyenne, les chercheurs qui publient sur la base de leurs propres données le font dans les 2 ans après avoir rendu les données publiques, contre 5 ans pour ceux qui travaillent sur des données ouvertes. En effet, il est moins aisé de travailler avec des données que l'on n'a pas produites.

Les auteurs concluent sur le plus clair des incitatifs : la reconnaissance et la propulsion de carrière des auteurs. Il convient d'informer les chercheurs du fait que les articles avec données ouvertes ont de meilleures citations, que les dépôts qui confèrent un DOI pérennisent les données et permettent de les citer. Certaines institutions et sociétés scientifiques créent des récompenses et incitations pour encourager l'open science (notamment la NASA, avec son projet Transform to Open Science).

L'objectif est que la publication de données et codes soit reconnue dans les carrières, recrutements, promotions et financements.

3.3.2 Caractère fonctionnel

Bien que l'aspect fonctionnel du code et sa qualité en tant qu'outil n'appartienne pas au FAIR en tant que tel, Trisovic et al. 2022 montrent que de bonnes pratiques FAIR dans la documentation permettent d'éviter la plupart des erreurs dans l'exécution du code. Stieglitz et al. 2020, ajoutent que les erreurs dans le code sont souvent dues à la méconnaissance des principes d'Open science des codeurs.

Au contraire, Vallet et al. 2022 prétendent que la description des dépendances dans la documentation est souvent insuffisante car la réutilisation est souvent entravée par l'environnement dans lequel s'exécute le code. Les auteurs présentent un outil libre parmi d'autres, Guix, qui permet d'empaqueter l'environnement complet pour le partager avec ses pairs et garantir la réexécution du code. Il existe également Docker. A noter que si le code source disparaît, les utilisateurs ne peuvent le compiler à nouveau. L'outil présenté est donc adossé à Software Heritage. Courtès et al. 2024 analysent la possibilité de récupération automatique des codes sources de l'archive.

3.3.3 Rôle des institutions

Enfin, il est important de soulever que plusieurs institutions ont déjà établi des directives ou des feuilles de route pour conduire à une meilleure gestion du code. On se référera notamment au rapport de la Research Software Alliance (Barker et al. 2024) qui rend compte des initiatives institutionnelles apparues depuis 2022 pour mettre en œuvre les Principes FAIR4S.

3.3.4 Importance de l'hébergement

Dans le prolongement du FAIR4S, Sonabend et al. 2024, tentent de compléter ces notions par les principes USE : FAIR-USE4OS (for Open Source Software). U, pour *User-centered*, implique que la documentation d'exécutabilité du code soit aussi détaillée que possible, tout en se réduisant au minimum requis, afin que l'utilisateur en fasse usage au prix d'un effort minimum. S, pour *Sustainability*, indique comment maintenir le code : sa communauté, sa fonctionnalité (bugs et nouvelles fonctionnalités) et ses finances. E, pour *Equitable*, énonce des principes généraux de logiciel éthique (accessible à tout utilisateur selon ses finances et la technique à sa disposition), sans exemple concret, sans critères précis et en admettant de nombreuses exceptions, ce qui réduit l'équité à une déclaration d'intention. L'article, peu documenté et pas exemplifié a néanmoins le mérite de souligner le point de vue des auteurs de code par rapport à celui de l'analyste FAIR que nous représentons : le simple export d'une version de code sur Zenodo ne suffit pas à faire vivre le code. Celui-ci doit être hébergé sur des sites web professionnels et des plateformes publiques.

3.3.5 Méthode de gestion

Les Principes FAIR n'étant pas une checklist opérationnelle pour les codeurs, Patel et al. 2023 s'essayaient à les formuler de manière pragmatique pour les professionnels. Dans cette mouvance, ils conçoivent une méthode de travail en 6 étapes à observer par un codeur avant, pendant et après le développement du software. Cette méthode est conçue pour la recherche biomédicale mais elle est pleinement applicable à tous types de Research software, pour peu qu'on applique des standards et ontologies spécifiques à la branche. La méthode met en œuvre les principes FAIR i) lors de la structuration du software et de son hébergement dans une forge (en vue de son partage avec la communauté des développeurs), ii) dans sa façon de la documenter, de documenter ses versions et de licencier le code (en vue de sa réutilisation) et iii) dans l'enregistrement du code dans l'archive Software Heritage.

3.3.6 Gestion de projet

Des outils sont proposés tels qu'un Software Management Plan (SMP) par Grossmann et al. 2024). La checklist fournie pour l'élaboration de logiciel est concise et complète. Dans le même esprit, Deekshita et al. 2024 donnent une feuille de route pour l'évaluation de la maturité d'un Research Software. Il s'agit là d'un outil de gestion de projet.

Abdill et al. 2024 proposent une méthode de modélisation et insistent sur sa nécessité dès les prémisses du projet. Celle-ci implique entre autres la redondance distributive, une licence et un code de conduite. Il s'agit d'une aide pour reporter l'utilisabilité des datas statiques et du code pour les biologistes.

4. Résultats de l'exploration

4.1 Méthode

Après constitution de la base de données et encodage, 56 articles et 81 mises à disposition ont été retenus.

Les résultats ci-dessous ont été obtenus selon les méthodes suivantes, qui sont détaillées dans le fichier Methodologie.md :

- Par des exports en CSV depuis AirTable, transformés en fichiers XLS avec tabulateurs. Les résultats ont été dupliqués par feuilles en fonction des résultats recherchés, puis triés et numérotés. La vérification visuelle et le comptage par formules s'imposaient, car une partie de l'encodage est fait en langage naturel, mots-clefs non structurés et disséminés dans différents champs.
- Par des formules sur la base de données, soit par les menus d'AirTable, soit en requêtant Omni, l'IA d'AirTable. Dans ce dernier cas, une vérification croisée a été faite visuellement, par des contrôles aléatoires et dans les tableaux Excel exportés. Les requêtes ont également été répétées par étapes. En effet, les erreurs d'Omni augmentent avec la complexité des requêtes.

4.2 Données des articles

4.2.1 Format

Par mesure de rigueur, les articles étant tous disponibles en PDF sur les sites des revues, ils ont été importés dans ce format dans notre dataset. Initialement, nous avons envisagé de les convertir au format HTML, plus pérenne que le PDF, puisqu'il ne s'agit pas d'un format propriétaire. Cette opération a été exécutée en ligne de commande dans le Terminal macOS, avec l'outil open source pdftohtml (suite Poppler, version installée via Homebrew 23.1.0). Les détails de la démarche sont donnés dans la méthodologie annexée.

De nombreuses erreurs sont apparues, notamment s'agissant des articles aux droits restreints, qui empêchent la conversion. Certains fichiers HTML ont gardé la mise en page originale en un fichier unique permettant une navigation linéaire (*A4-like*), mais la plupart étaient fragmentés, chaque section étant exportée dans un fichier séparé et accessible via un sommaire cliquable (HTML squelettique). Le temps à consacrer pour la vérification de la complétude des fichiers aurait été disproportionné par rapport à ce travail. Notre étude se borne en effet à l'évaluation des principes FAIR des publications. Les formats de publications font partie des nombreux critères que nous analysons.

La démarche consistant à rendre FAIR des documents publiés par d'autres est un jeu sans fin. Nous préférons nous concentrer sur des recommandations. Cet écueil met en évidence de l'interdépendance existant dans la recherche ouverte.

Quoiqu'il en soit, un certain nombre d'articles sont disponibles en XML ou EPUB sur la page de l'éditeur. Ils sont relevés ci-dessous :

Tableau 1 : Formats d'articles proposés par les revues scientifiques

Revue	Exclus. OA	PDF	XML	Metadata XML	EPUB
Academic Pediatrics		✓			
AHA Journals		✓			✓
Biodiversity Data Journal	✓	✓	✓		
Biotechniques (Taylor & Francis)		✓			
Cambridge University Press		✓			
Frontiers	✓	✓	✓		✓
IOP Science		✓			
JMIR Publications	✓	✓	✓		
MDPI	✓	✓	✓		✓
Nature		✓			
Peerj.com	✓	✓	✓		
Oxford Academic		✓			
PLOS One	✓	✓		✓	
PRO ASTRO (Elsevier)		✓			
Sage Journals		✓			
ScienceDirect (Elsevier)		✓			
Springer Nature		✓			
The BMJ		✓			
Transnational and Clinical Pharmacology		✓			
Wiley		✓			

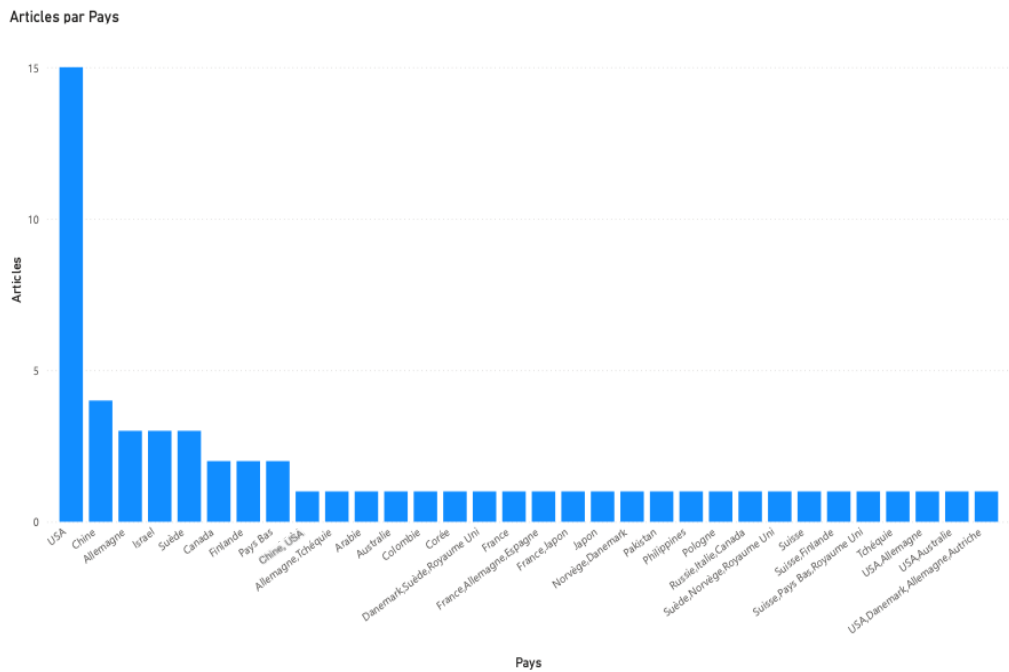
(Formats et Revues par Article)

On constate que la plupart des revues, y compris celles de renommée, ne prévoient (ou n'exigent pas ?) de format pérenne tel que XML. La revue PLOS One ne diffuse qu'un document XML des métadonnées de l'article. Seules Biodiversity Data Journal, Frontiers, JMIR Publications, MDPI, Peerj.com, proposent des formats plus pérennes.

4.2.2 Provenance

À titre informatif, on a relevé les pays des institutions de recherche au sein desquelles les auteurs ont publié. Les États-Unis sont prépondérants (19 articles sur 56, soit 34%). De nombreux pays ne sont auteurs que d'un (1) article, si bien que cette donnée, sur un échantillon modeste, n'est pas exploitable. Elle est illustrée ici à titre indicatif.

Figure 4 : Provenance des institutions à l'origine de la recherche

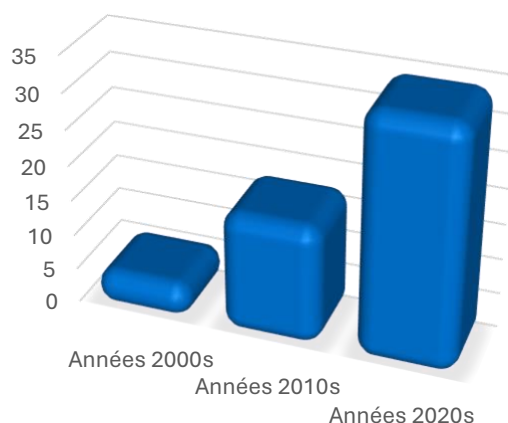


(Pays de l'institution par Article)

4.2.3 Date de publication

Les 56 articles s'étendent dans le temps comme suit : 5 articles datent des années 2000 (2001 à 2007), 17 des années 2010 (2013 à 2019) et 34 des années 2020 (2020 à 2025).

Figure 5 : Date des articles



(Article par Date)

A noter que ni la date ni la langue des articles (l'anglais) ne procèdent d'un choix mais résultent des requêtes effectuées dans PubMed. Il était attendu que les chercheurs en biomédecine publient articles et logiciels en anglais.

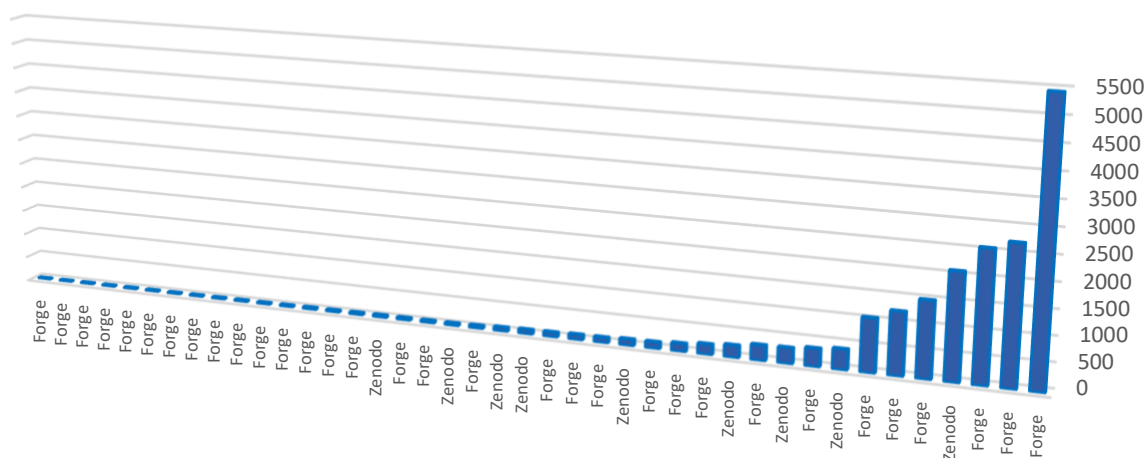
4.3 Fréquentation du code

Les variables indiquant la fréquentation du code ont été relevées. Elles sont présentées avant les Principes FAIR car, même si le succès d'un code contribue à sa trouvabilité et à sa réutilisabilité (une bonne fréquentation donne « confiance » dans le code), il ne représente pas un critère de pérennité et n'apporte pas de qualité FAIR en soi.

GitHub donne les statistiques suivantes : *stars (likes)*, *watching* (suivi), *forks* et *pull requests* (demandes de fusion, ouvertes et fermées). Les *branches* n'ont pas été relevés. Malheureusement, les variables de « succès » (*downloads*, *like*, *supporters*...) dans les diverses forges et sur Zenodo ne relèvent pas les mêmes données, si bien que la mesure du succès n'est que peu révélatrice. En outre, le succès provenant de statistiques des plateformes, les autres mises à disposition sont exclues de cette estimation.

A des fins d'illustration, un champ additionnant les critères de succès a été créé pour donner une idée de la fréquentation des codes du corpus. L'information la plus importante est que les codes du corpus sont d'envergure très variable (d'un seul contributeur à une grande communauté).

Figure 6 : Succès du code



(Somme Succès du code par Mise à disposition)

Ne figurent pas sur le graphique : 29 mises à disposition avec un succès=0 et 11 mises à disposition dont le succès est inférieur à 1000. Sur les 52 mises à disposition représentées ici, 7 codes ont plus de 1000 points de succès dont un (1) a 5441 points. Les codes sur Zenodo ont entre 2090 et 38 points.

4.4 F - Findable

4.4.1 Mise à disposition

Le choix de la mise à disposition est présenté ici sous -F- mais il impacte tous les Principes FAIR.

Plateformes de développement (forges)

45 enregistrements correspondent à des codes hébergés sur GitHub. Ces enregistrements proviennent de 38 articles (un auteur pouvant diffuser plusieurs éléments de code séparément en relation avec un même article : 37964216, 38951670, 38039154, 36699385 et 38776539 ont 2 enregistrements sur GitHub, 38032878 en a 3).

En d'autres termes, sur 81 mises à disposition, environ 55% concernent GitHub. Sur 56 articles, 38 ont leur code sur GitHub, soit 68%.

Le code d'un article se trouvant sur GitHub (16211538) est également disponible sur GitLab et Zenodo. En outre, le code d'un (1) article (28096085) est disponible exclusivement sur BitBucket.

Ceci porte le nombre d'articles dont le code est sur une plateforme de développement à 39, soit 70% des articles.

Plateformes de dépôt

Parmi les 45 codes déposés sur GitHub, 7 le sont également sur Zenodo. L'un d'eux (1) est celui qui est disponible sur GitLab.

Seuls 2 articles sont à disposition exclusivement sur Zenodo (39682433 et 39314734).

En tout, les mises à dispositions sur Zenodo sont donc au nombre de 9, soit 16% des 56 articles.

Plateformes de distribution de packages

Un (1) article (24618462) a son code disponible sur CPAN. Un autre (1), (38032878), disponible sur GitHub, l'est également sur CRAN, Pypi et Anaconda. Un troisième (1) (36699385), disponible sur GitHub, l'est aussi sur Bioconductor. Contrairement aux précédentes, cette plateforme pour le langage R est spécialisée en bio-informatique.

Au total, 3 articles (4%) hébergent leur code sur une plateforme de distribution de packages, dont un exclusivement.

Site académique

Le code d'un seul article (1) (15884679) est disponible sur ComplexComputation.org, le laboratoire de recherche logicielle de l'université d'Iowa (2%).

Pratiques peu conformes

Hébergement sur PubMed : 4 codes, provenant de 4 articles différents, sont disponibles exclusivement sur la page PubMed de l'article. Les 4 articles concernés représentent 7%.

Sites ou clouds propriétaires : deux articles ont leur code sur des sites distribués par Google et un (1) sur un site propriétaire dédié au logiciel (3 articles en tout, soit 5%).

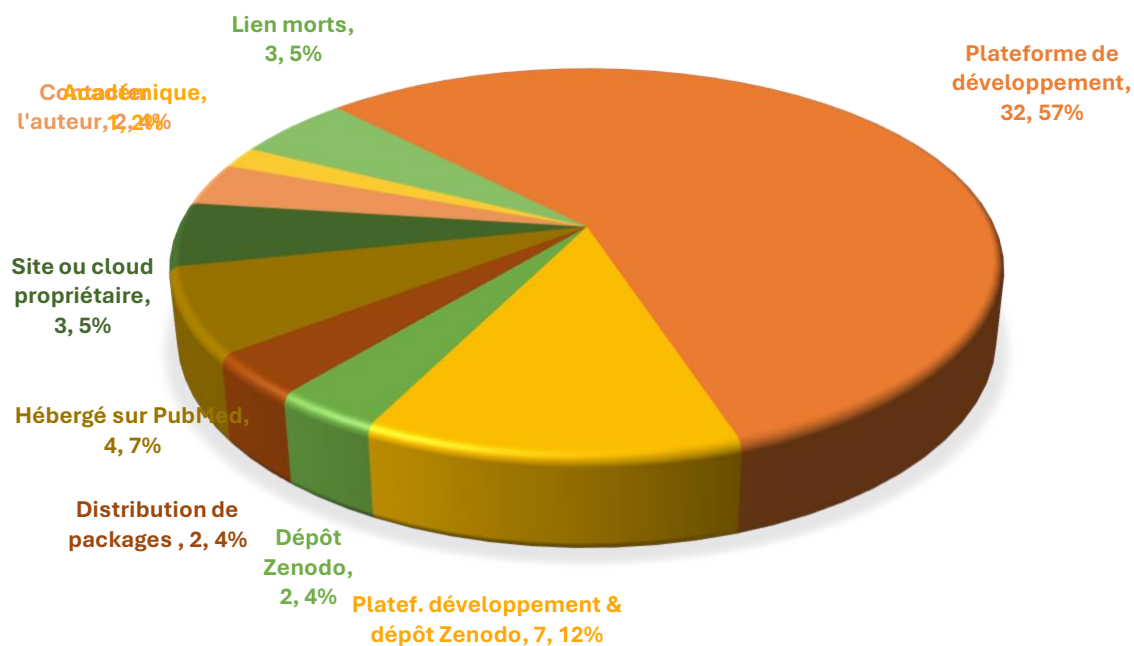
Contact direct : 2 codes (correspondant à 2 articles) sont disponibles en contactant l'auteur (4%).

Erreur : 3 articles (5%) ont des liens morts (11%).

Les 4 dernières sections (éditeur, sites clouds et propriétaires, contact direct et erreur) représentent des mises à disposition peu FAIR et non adaptées au logiciel. Elles sont au nombre de 15 et correspondent à 27% des articles.

Vue d'ensemble de la mise à disposition

Figure 7 : Choix de la mise à disposition du code par les auteurs



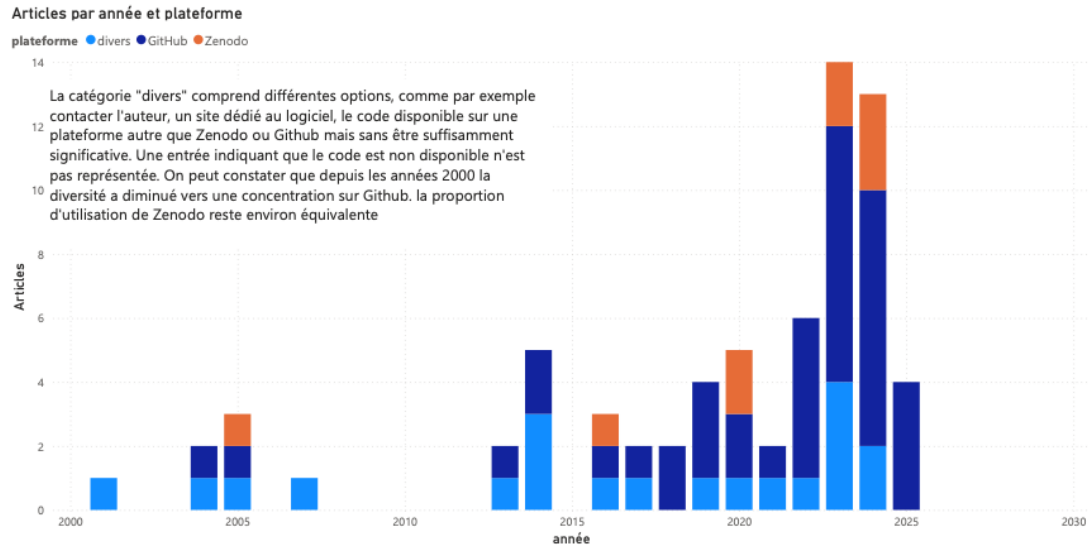
(Mise à disposition par Article)

Au total, 71% du code est déposé au moins sur une plateforme de développement ou sur le dépôt Zenodo.⁹ 68% du code est géré sur une plateforme de développement.

⁹ Sont prises en compte l'ensemble des mises à disposition (y compris celles, multiples afférentes à un même article). C'est le cas des figures ci-après, sauf mention contraire.

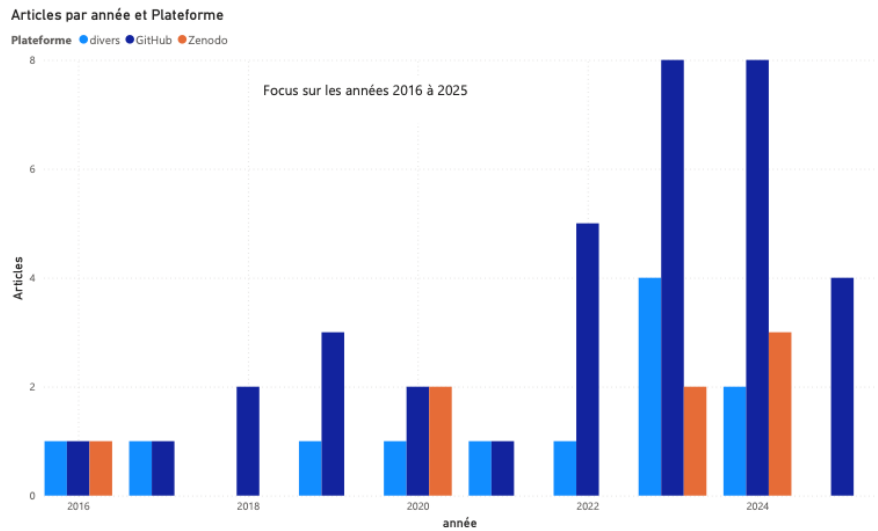
Par ailleurs, on constate que le choix entre plateformes et autres mises à disposition évolue avec le temps :

Figure 8 : Choix de mise à disposition de 2001 à 2025



(Mise à disposition par Article par Année dès 2001)

Figure 9 : Focus sur la mise à disposition de 2016 à 2025



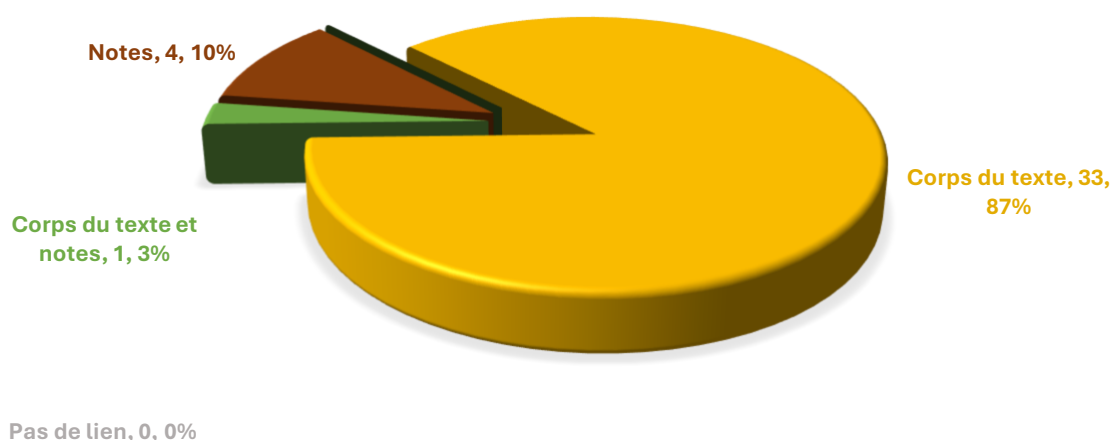
(Mise à disposition par Article par Année dès 2016)

4.4.2 Lien vers le code

De l'article au code

Sur 56 articles, 38 ont une publication du code sur le net. Toutes donnent le lien vers le code dans le corps de l'article ou dans ses notes : 33 dans le corps de l'article, 4 uniquement dans les notes et une (1) aux deux endroits. Dans tous ces cas, le lecteur n'a pas besoin de se référer à la page d'un éditeur ou de PubMed, ni d'aller à la pêche aux informations pour localiser le code.

Figure 10 : Emplacement du lien vers le code



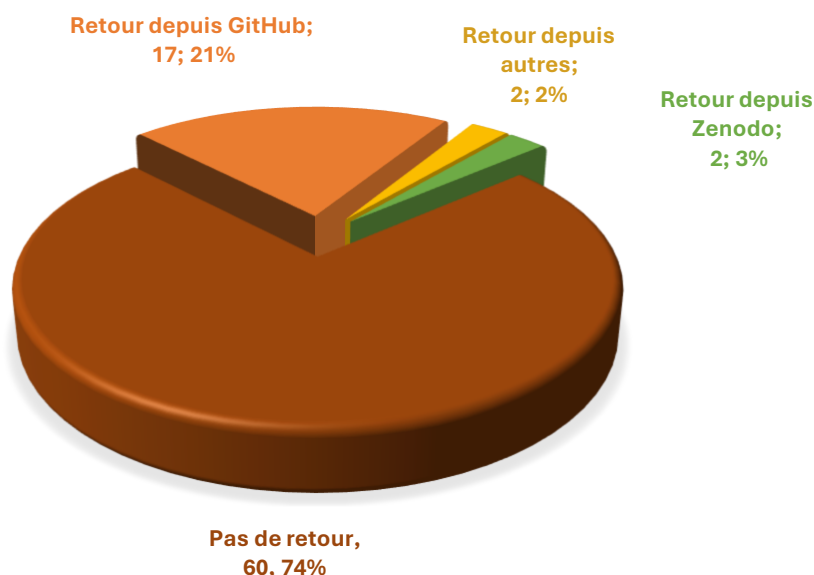
(Emplacement du lien par Article)

Du code à l'article

La navigabilité est également facilitée lorsque les métadonnées du code renvoient à l'article. Ceci peut se faire dans les indications de provenance du ReadMe par exemple. Sur les 81 mises à disposition, 21 codes contiennent un lien de retour vers l'article. Ces 21 codes correspondent à 17 articles, soit 30% du corpus.

Ils se distribuent comme suit : un (1) code depuis une plateforme de distribution de packages (bioconductor.org), un depuis ComplexComputation.org (laboratoire de recherche de l'Université d'Iowa), 2 depuis Zenodo et 17 depuis GitHub (correspondant à 14 articles).

Figure 11 : Lien de retour vers l'article



(Retour de lien par Mise à disposition)

4.4.3 Qualité des métadonnées

La première des conditions de riches métadonnées est la publication d'un ReadMe ou d'un fichier semblable. Sur les 81 mises à disposition, 48 comprennent un ReadMe (59%), 32 n'en montrent pas. 38 articles sur 56 (68%) ont assorti leur code d'un ReadMe dans l'une des mises à disposition au moins. 19 articles n'ont pas publié de ReadMe.

Sur les 9 articles déposés sur Zenodo, 2/3 ont un ReadMe (6 sur 9). (Ces chiffres ne préjugent pas de la qualité du ReadMe).

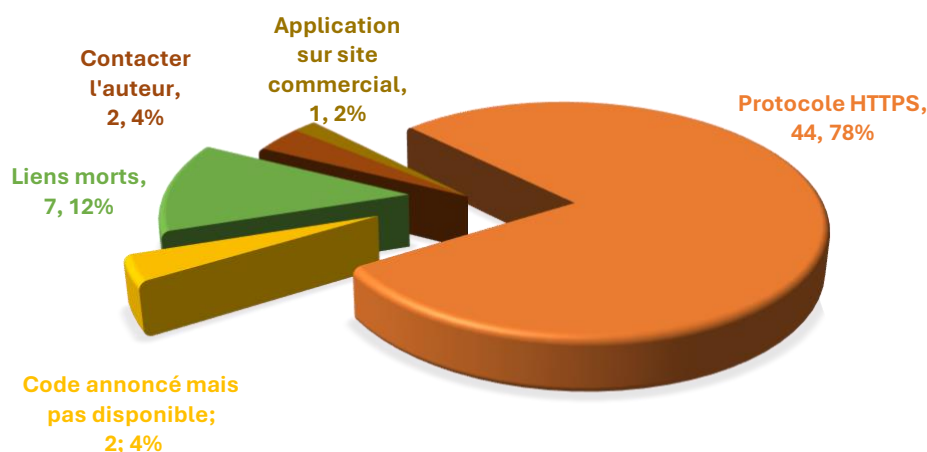
Une analyse structurée du contenu et de la structure de ReadMe a été effectuée de manière empirique. Elle est présentée et analysée dans la partie Analyse critique des résultats, sous le Principe -R- (5.5.1).

4.5 A - Accessible

S'agissant des protocoles de récupération, le code présent sur les plateformes (forages, dépôt et distribution de packages) : est récupérable via une page web (protocole HTTPS), à savoir universellement accessible et gratuite (44 articles).

7 articles contiennent des liens morts vers le code, 2 l'annoncent sans le mettre à disposition et 2 sont disponibles en contactant l'auteur. Au total, ce sont 11 articles qui ne respectent pas le principe -A- (20%).

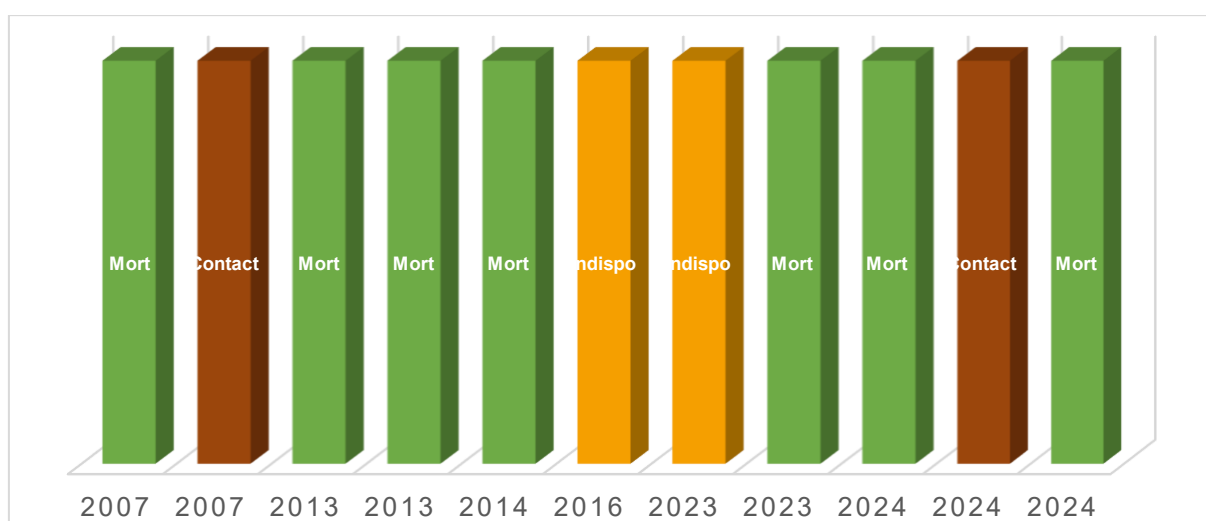
Figure 12 : Protocole d'accès au code



(Protocole d'accès par Article)

Les dates de publication des 11 articles dont le protocole n'est pas FAIR sont réparties comme suit entre 2001 et 2025 :

Figure 13 : Protocoles d'accès restreint au code



(Sélection de Mise à disposition (non FAIR) par Date de l'article)

4.5.1 Authentification et Autorisation

Les codes présentés sur GitHub, Zenodo et les plateformes de distribution de packages permettent d'authentifier et autoriser les utilisateurs voulant accéder au code, puisque ces procédures sont prévues par les deux plateformes.

S'agissant des données disponibles dans l'article, l'authentification et l'autorisation dépendent des conditions dans lesquelles sont publiés les articles. Les autres cas sont à étudier un à un.

4.5.2 Pérennité des métadonnées

La pérennité (*persistence*) des métadonnées après la mort du software n'est pas garantie par GitHub, puisque la plateforme n'est pas pérenne et que le détenteur du compte garde la possibilité de supprimer la page du code. Il en est de même pour les plateformes de distribution de packages.

En revanche, Zenodo attribue un DOI au code et interdit au détenteur de retirer la page. Ainsi, lorsque le code déposé n'est plus viable, les métadonnées qui l'accompagnent demeurent utilisables.

S'agissant des autres cas, la pérennité des métadonnées n'est garantie qu'à condition que le code ait un identifiant pérenne, ce qui n'est pas le cas dans notre corpus.

4.6 I - Interoperable

4.6.1 Standards

Les variables concernant l'interopérabilité sont peu nombreuses dans ce travail car la récolte systématique des ontologies et standards de la communauté en dépasse le cadre. Il en est de même concernant les métadonnées qui renverraient aux dépendances du code.

4.6.2 Langages

On peut en revanche relever une partie des informations au sujet des langages, formats et extensions.

Langages

GitHub relève de façon automatique les langages utilisés dans le code, sur la base du contenu et des extensions (par exemple R, 70% et C++, 30%). À noter que lorsqu'un langage n'apparaît que rarement dans un code, *Linguist*, l'outil statistique de GitHub, n'en tient pas compte. Ceci peut être corrigé par le détenteur de la page.¹⁰

Extensions

s'agissant des softwares mis à disposition par d'autres biais, seules les extensions ont été relevées de manière manuelle sur la simple observation de l'arborescence du code, ou sur la foi des indicatifs des auteurs de l'article.

Les informations proviennent tant des langages que des extensions. Les proportions ci-dessous sont purement indicatives.

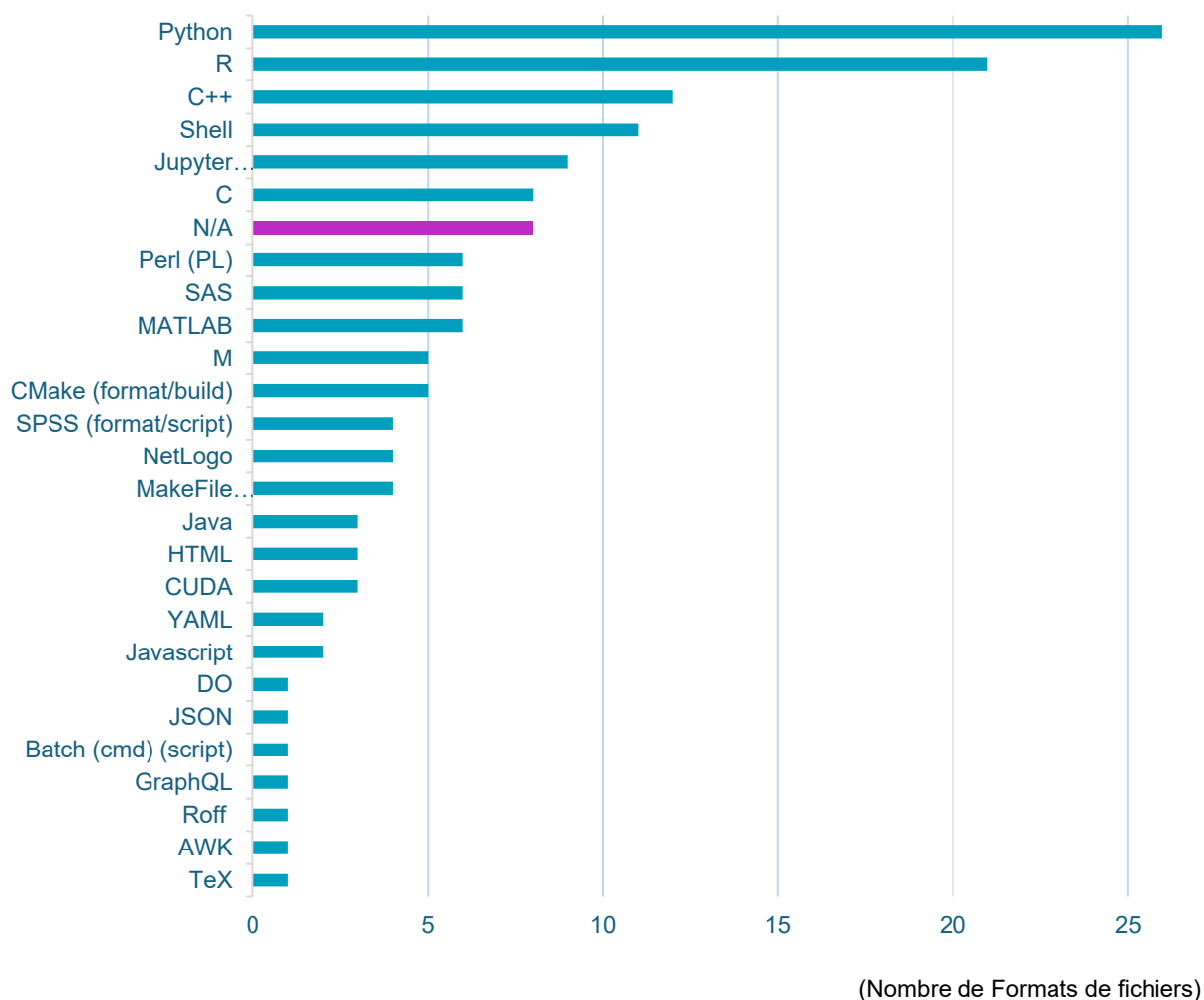
¹⁰*About repository languages*. Documentation de GitHub. Disponible à l'adresse <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-repository-languages?versionId=free-pro-team%40latest&productId=get-started&restPage=learning-about-github%2Cgithub-language-support>. (Consulté le 13 janvier 2026).

Les langages et extensions identifiés sont les suivants :

AWK, Batch (Windows CMD), C, C++, CMake, CUDA, DO, GraphQL, H, HTML, Java, JavaScript, JSON, Jupyter Notebook, M, Makefile, MATLAB, NetLogo, Perl, Python, R, Roff, Ruby, SAS, Shell, SPSS, TeX et YAML.

8 mises à disposition sont sans indication.

Figure 14 : Langages/formats/extensions des fichiers des codes



On relève encore que des codes sont mis à disposition en format image PDF ou PNG) dans les suppléments de la page PubMed ou dans le corps même de l'article, ce qui contrevient aux principes FAIR :

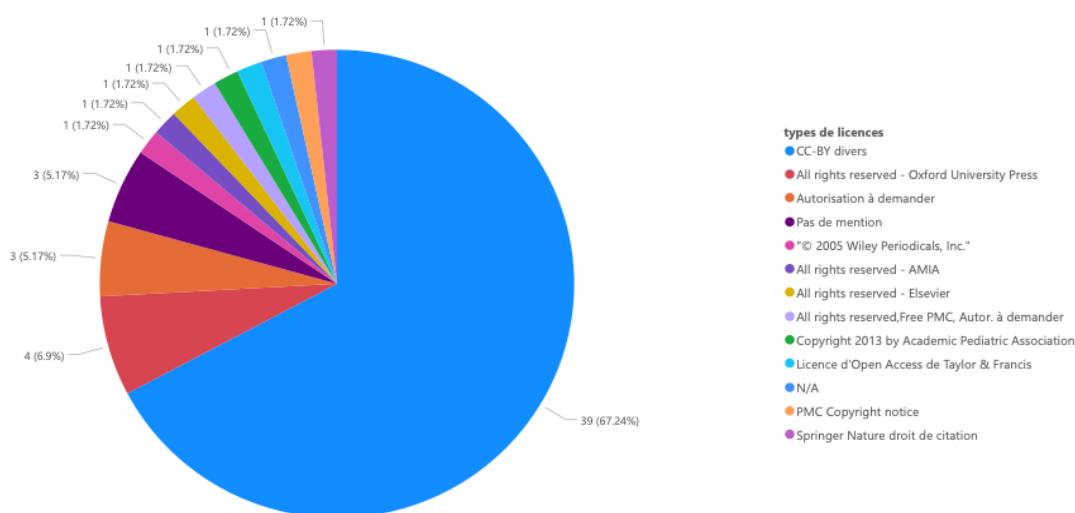
4 articles présentent le code sous forme de PDF OCRisé et donc copiable dans un éditeur de code (26858290, 38090166, 31763386 et 33425800) et 2 sous forme d'image plate (33936490 et 38237907). Nous reviendrons sur les raisons de ces 6 exceptions dans l'analyse.

4.7 R - Reusable

4.7.1 Licence des articles

Les licences de publication des articles sont variées, mais on constate que la majorité des articles (39), est publiée sous une licence Creative Commons (quelle que soit sa version). Ceci représente 67%.

Figure 15 : Licence des articles



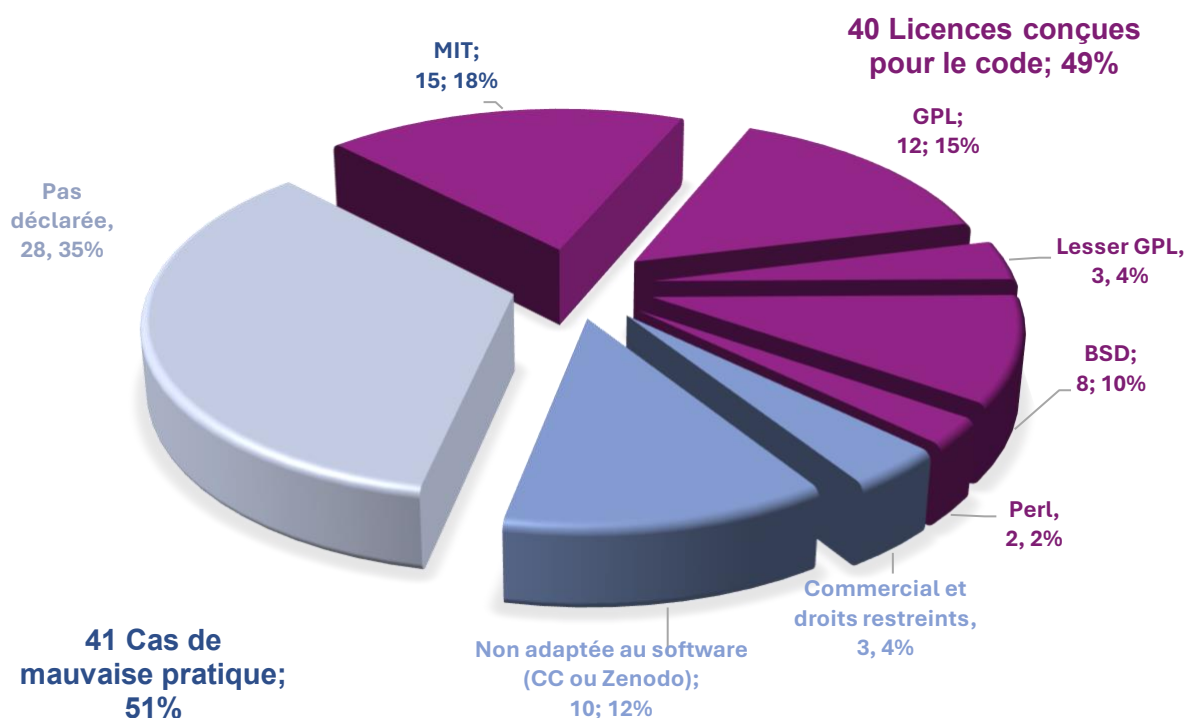
(Articles par Licence)

4.7.2 Licences du code

Sur 81 mises à disposition, 40 codes (49%) ont une licence adaptée à la forme logicielle, c'est-à-dire qui fixe le sort des dépendances et de la réutilisation de tout ou partie du code dans de nouveaux logiciels. Les licences choisies dans le corpus sont MIT, BSD, GPL, Perl.

10 auteurs d'articles ont choisi une licence qui n'est pas adaptée à l'objet code : 9 ont déclaré une licence Creative Commons et un a déclaré une licence générale de Zenodo dite « Open ». 10 articles/mises à disposition ont une licence non adaptée au code (12%). 3 codes ont des droits restreints ou commerciaux (4%) et 28 n'ont aucune licence déclarée (35%).

Figure 16 : Licence déclarée par mise à disposition



(Licence par Mise à disposition)

4.7.3 Correspondance entre licences

On peut relever certains liens entre le choix de la licence déclarée pour un article et celle déclarée pour son code.

Analyse par article : 17 articles publiés sous licence Creative Commons ont un code sans licence déclarée.

Analyse par code : Sur les 10 cas de licences non adaptées au code, 9 correspondent à un article avec licence Creative Commons. Le 10^{ème} (32096823) conditionne les droits d'usage de l'article à la prise de contact avec l'auteur. Il s'agit de l'article dont le code est déposé sur Zenodo assorti d'une licence « Zenodo Open ».

Les codes et mises à disposition dont la licence est appropriée au logiciel sont au nombre de 40. Parmi ceux-ci, 28 sont liés à des articles avec licence libre (26 CB-BY et 2 autres licences).

Parmi les pratiques peu FAIR, 2 déclarent une licence Creative Commons ND (No-Derivatives : droit d'attribuer sans modifier), 5 demandent de contacter l'auteur, 4 ont des droits restreints et un (1) seul article n'a pas de licence déclarée. En résumé, sur les 40 codes correctement licenciés, 10 articles ne respectent pas les principes FAIR en matière de licence.

Les codes sans déclaration de licence (29) sont liés tant à des articles sous Creative Commons que sous droits restreints, no-derivative, contacter l'auteur ou sans licence. La majorité d'articles sous Creative Commons (18) correspond à la grande proportion du corpus et n'est donc pas pertinente.

On peut tenter d'établir un lien entre la connaissance qu'ont les auteurs des enjeux de la licence de leur article d'une part, et du code d'autre part. On a donc cherché, pour chaque code correctement licencié, quelle licence avait été attribuée à l'article correspondant :

Licence du code	Licence de l'article
BSD	→ 7 articles avec Creative Commons (100% FAIR)
CC-BY	→ 9 Creative Commons et 1 « contacter l'auteur » (à noter qu'aucun code n'est publié sous Creative Commons No-derivative) (90% FAIR)
Zenodo « Open »	→ 1 « contacter l'auteur » (Pas FAIR)
GPL	→ 7 Creative Commons ou Licence libre 1, 3 « contacter l'auteur » et 1 licence non déclarée (64% FAIR)
GPL lesser	→ 3 droits restreints (Pas FAIR)
MIT	→ 12 Creative Commons, 2 « contacter l'auteur » 1 droit restreint (80% FAIR)
Perl	→ 2 Creative Commons No-derivative (Pas FAIR)

En résumé, les codes licenciés sous BSD, Creative Commons CC-BY, GPL et MIT, qui sont les licences adaptées au code et conférant suffisamment de droits pour être FAIR, proviennent en grande partie d'articles ayant des licences adéquates.

4.7.4 Réutilisation effective

Forme exécutable

Concernant la forme de mise à disposition du code, la quasi-totalité représente du code source, et non des fichiers binaires. Seuls 2 logiciels (2,5%) sont mis à disposition sous forme exécutable : Eeglab (15102499), sur son site dédié au logiciel open source et Kubios (24054542), application commerciale disponible dans l'Apple Store depuis son site.

Ce résultat indique que les auteurs d'articles choisissent presque systématiquement la forme du code source non compilée.

Forme empaquetée

Comme indiqué plus haut, une solution pour garantir la réutilisation du code source consiste à encapsuler l'ensemble de l'environnement d'exécution dans une image Docker, y compris les outils externes au code. Une recherche systématique n'a pas été menée pour trouver des cas d'encapsulation du code dans le corpus, et ils constituent donc une forme de mise à disposition non répertoriée. L'un d'eux au moins (15102499) présente cette solution. Le code est versionné et ses versions sont gelées sur [hub.docker.com](https://hub.docker.com/r/ncbi/blast/tags) sous forme d'images docker¹¹. L'emploi de l'outil Docker (logiciel gratuit dans une certaine mesure) est ensuite nécessaire pour télécharger l'image et l'exécuter.

¹¹ Il s'agit du logiciel NCBI/Blast, disponible à l'adresse <https://hub.docker.com/r/ncbi/blast/tags>.

Caractère fonctionnel

Le temps imparti pour cette étude n'a pas permis de procéder aux tests initialement envisagés concernant la qualité du code. Après réflexion, il n'a pas été jugé pertinent de procéder à des tests sur quelques codes pertinents du corpus, car ceci dépasse la mission de l'assesseur FAIR en tant que conseiller en matière de publication.

5. Analyse critique des résultats

5.1 Articles

5.1.1 Format des articles

Comme constaté plus haut, la plupart des revues ne prévoient pas – ou du moins n'exigent pas – de format pérenne tel que XML pour la diffusion des articles qui leur sont soumis. Dans notre corpus, seules les revues dédiées strictement à l'Open Access, à savoir Biodiversity Data Journal, Frontiers, JMIR Publications, MDPI et Peerj.com.

S'agissant des géants de la publication mondiale tels qu'Elsevier ou Wiley, on peut admettre que leurs recherches en matière de pérennisation des données ne soient pas poussées car un tel investissement ne se justifie probablement pas selon leur modèle d'affaires.

Il est en revanche étonnant qu'une revue consacrée à l'Open Access et soutenue par un organisme à but non lucratif comme PLOS One n'ait pas abordé la question du format de ses publications et ne diffuse en XML que les métadonnées de soumission de l'article. Il serait souhaitable que les organismes responsables de la publication en ligne mettent à disposition de leurs auteurs des modèles HTML incluant la gestion automatique des figures et des métadonnées.

5.2 F - Findable

5.2.1 Pérennité du dépôt

Il est exigé que les données statiques liées à un article scientifique soient déposées dans un dépôt tel que Zenodo. Toutefois, en matière de logiciel, 70% des articles de notre corpus hébergent leur code sur GitHub. Il convient d'en tenir compte lors de l'élaboration de critères FAIR.

L'aspect qui manque le plus sur une forge est la pérennité : la plateforme ne garantit pas de conservation à long terme puisqu'elle est la propriété de Microsoft. A fortiori, elle ne fournit pas d'identifiant pérenne, ce qui contrevient au principe -F-, contrairement à Zenodo. Pour rappel, seuls 7 codes de notre corpus satisfont au principe -F- sur la question du DOI. Ce sont ceux qui sont déposés sur Zenodo. En effet, le but d'une forge étant le développement par la communauté, le code qui s'y trouve est modifié, versionné. Contrairement à un dépôt Zenodo, la page GitHub d'un code peut simplement être effacée par son auteur.

S'agissant du choix quasi unanime de GitHub et Zenodo, il convient de proposer deux approches. D'une part, Wilkinson et al. 2016 mettaient en garde contre les plateformes généralistes, qui n'ont que peu d'exigences concernant la standardisation des données et des

descripteurs. Ils citaient pour exemple le cas d'un chercheur qui voudrait comparer ses données locales sur des agents pathogènes à d'autres exemples connus et examiner leurs comportements pendant une infection. En l'absence d'un dépôt spécialisé pour la polyadénylation différentielle, il serait souhaitable que les données des recherches précédentes, hébergées sur une plateforme généraliste, comprennent au moins les filtres spécifiques à l'étude (espèce, gène concerné, type de tissu, conditions d'analyse, par exemple durant l'infection, etc.). En l'absence de standardisation, la compilation prendrait des semaines. (Wilkinson et al. 2016, p. 2)

D'un autre côté, le succès des grandes plateformes généralistes et ouvertes à l'échelle mondiale comme GitHub, dont les codeurs sont captifs, peut jouer en la faveur d'une pérennisation du code : un hébergement sur une plateforme trop spécialisée (« niche ») présente le risque d'échapper aux moissonneurs du web que sont Web Archives et Software Heritage (Escamilla et al. 2023, pp. 3 et 12). La trouvabilité s'en trouve affectée pour les humains comme pour les machines. Contre toute attente, la multitude et la spécialisation des plateformes d'hébergement du code ne plaident pas en la faveur de la pérennisation. 10 ans après Wilkinson et al. 2016, le choix d'un GitHub semble plus FAIR que celui des plateformes spécialisées. On peut parler d'hébergement en silo ou de longue traîne des plateformes d'hébergement.

Néanmoins, on constate que l'exigence de riches métadonnées est respectée, non pas par le choix des infrastructures, mais par l'adoption de standards et d'ontologies spécifiques à chaque branche. Il existe ainsi des plateformes telles que fairsharing.org, qui fournissent des standards de métadonnées, ontologies contrôlées et schémas de données par la communauté scientifique, ainsi que bioschemas.org, qui étend aux ressources scientifiques le vocabulaire structuré de schema.org.

Au vu de ce qui précède, on comprend que le respect des Principes FAIR n'est pas constitué de critères objectifs binaires (FAIR ou non FAIR). Une pesée des intérêts doit être faite par le chercheur lors de son SMP pour satisfaire la réutilisabilité de son code.

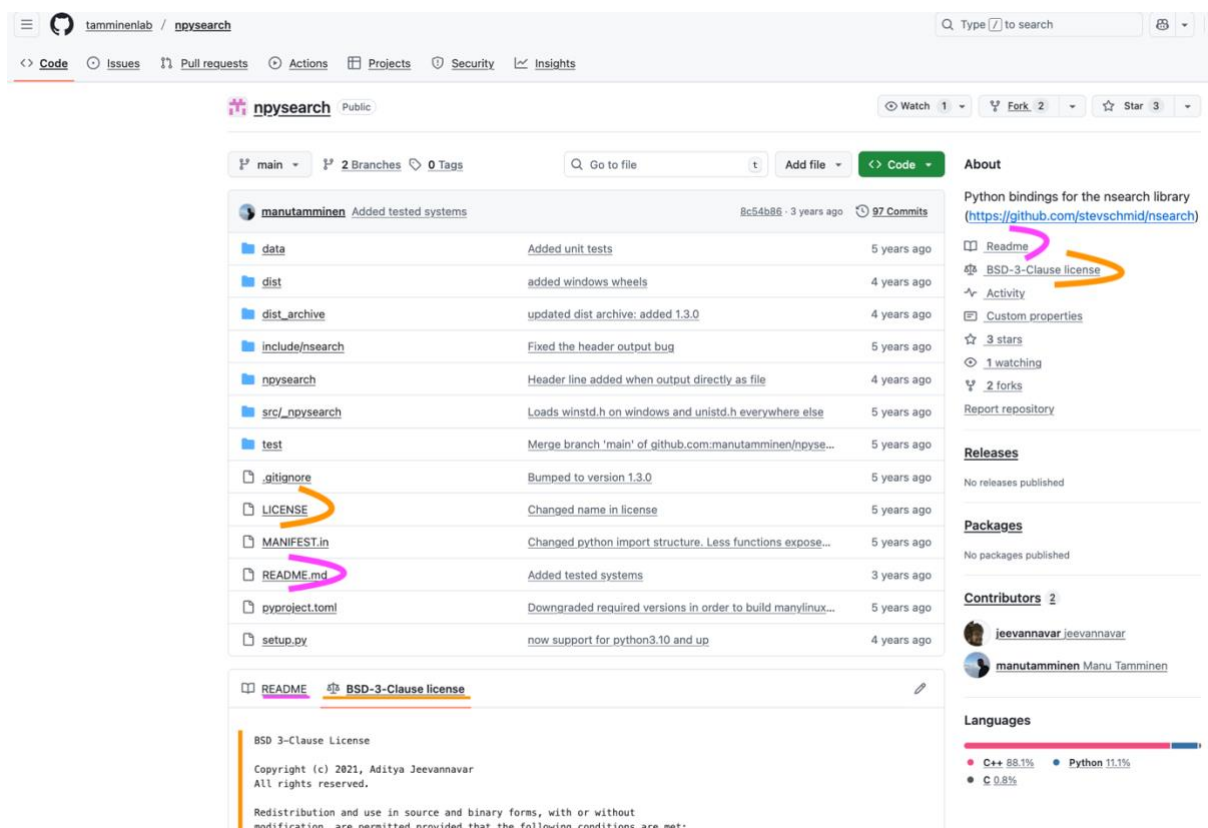
5.2.2 Structuration des métadonnées

Quoiqu'il en soit, le peu de code trouvé dans le dépôt Zenodo révèle un manque d'adaptation de cette plateforme à l'objet code, du moins dans l'esprit des chercheurs. Si l'API qui mène à Zenodo est fonctionnelle, le dépôt manuel de code sur Zenodo n'est qu'une capture d'un état du code et ne permet pas une lisibilité par l'humain. La navigabilité (c'est-à-dire -F- appliqué aux humains) est meilleure sur une forge. Un dépôt ne donne pas l'information sur la vivacité du code, sa structure, son historique, son succès, contrairement à la navigation sur une forge.

En outre, la structure des informations que l'utilisateur peut renseigner dans un page GitHub (ou autre GIT) reflète la méthodologie appliquée pour le respect des principes FAIR. À titre d'exemple, la forge demande que soient renseignées les informations du `README` ou de la `LICENCE` : celles-ci sont visibles dans un fichier éponyme, mais également sur les badges ReadMe et Licence (en haut à gauche de la capture ci-dessous). La qualité des métadonnées est immédiatement compréhensible et donne une idée sur l'aspect sérieux du contenu, sur la confiance que l'on peut lui accorder.

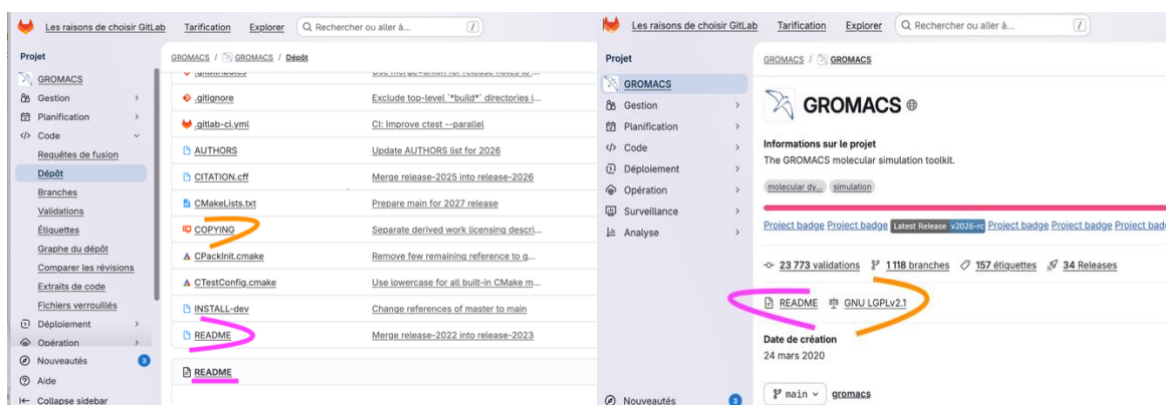
Dans ce contexte, les statistiques de fréquentation et de participation au code (son succès auprès de la communauté) sont également visibles au premier coup d'œil. Cette donnée participe au principe de provenance de -R- dans une certaine mesure.

Figure 17 : Exemple de page GitHub



(<https://github.com/tamminenlab/npyssearch>)

Figure 18 : Exemple de page GitLab



(<https://gitlab.com/gromacs>)

De même, l'arborescence des fichiers est visible sur une forge permet au premier coup d'œil et permet d'évaluer si l'organisation du dépôt respecte les principes FAIR de classement des fichiers. (Nous n'avons toutefois pas relevé cette donnée, qui va au-delà de la simple analyse FAIR).

On note encore les informations telles que les contributeurs, les langages utilisés, les branches et tags. Dans tous les cas, la majorité des auteurs considère que l'hébergement sur une forge est essentiel pour le respect des principes FAIR (notamment Patel et al. 2023, p. 4). Compléter les rubriques d'une forge dès la première étape conditionne la qualité du software

Enfin, bien que cet aspect ne soit pas couvert par cette étude, on peut noter que la mesure finale de pérennisation consiste à archiver à long terme son code dans une archive publique dédiée aux logiciels, telle que Software Heritage. Celui-ci moissonne le web à la recherche de code, mais un codeur peut également « pousser » son code pour le diffuser sur cette archive de manière volontaire. À noter qu'une fois le code archivé sur Software Heritage, aucun update n'est nécessaire pour les prochains *commits* car l'archive moissonne régulièrement le site et archive les versions successives automatiquement. (Patel et al. 2023, p. 7)

5.2.3 Qualité des métadonnées

Comme dit plus haut, les données concernant la qualité et le contenu de la documentation est présentée et analysée dans le même chapitre 5.5.1, pour des raisons de commodité.

5.2.4 Granularité de l'identification pérenne

L'identifiant pérenne d'un code doit être émis à chaque release (F1.1). Lamprecht et al. (2022, p. 43) ont adapté le Principe F1.2 au software en ajoutant l'exigence d'une identification pérenne de chacun des composants du code. Ceci implique d'identifier chaque niveau de granularité du code.

Les niveaux de granularités sont définis du plus général au plus détaillé comme suit par Chue Hong et al. (2022, p. 16) : Projet, versions du projet, modules, sous-modules, snapshots, releases, *commits*, répertoires, dossiers et fragments de code. Dans les faits, on peut se demander si une telle démarche peut être raisonnablement exigée des chercheurs : la multiplication des dépôts et la systématique du processus constituent une charge supplémentaire pour les chercheurs et comportent des risques d'oubli. À l'appui de cette réserve, on notera que le Principe -F- ne prescrit pas le niveau de granularité auquel attribuer des identifiants. Celui-ci est laissé à la discrétion des chercheurs selon les standards applicables dans leur domaine (Ibid., p. 7).

5.3 A - Accessible

5.3.1 Protocole d'accès

On constate que les 11 articles dont le protocole ne satisfait pas les critères FAIR ne concernent pas uniquement des publications anciennes, dont on comprendrait que le lien soit mort ou que les principes FAIR n'aient pas encore été connus de leurs auteurs : 5 d'entre eux datent de 2023 et 2024. Ils sont émis par Stockholm Bioinformatics Center, Université de Macao, Entreprise privée Huawei & Université de Moscou, Lakehead University (Canada) et Hopp Children's Cancer Center (Allemagne).

5.4 I - Interoperable

5.4.1 Standards

Comme indiqué, la collecte des standards et schémas du corpus devraient faire l'objet d'une recherche suséquente. Les remarques concernant ces standards sont néanmoins présentées dans le chapitre concernant le Principe -R-. (5.5)

5.4.2 Formats image

Comme relevé dans l'étude exploratoire, 5 articles contiennent le code en format image (PDF plat ou OCRisé) dans le corps du texte. Ceci semble une infraction au Principe -I-, car ce format ne permet pas l'extraction ni l'automatisation du code. Mais à y regarder de plus près, certains cas sont justifiés.

À titre illustratif

Souvent les codes sont des fragments de code dans un langage courant, tel que -R-, à titre d'explication. Ceci peut relever d'une volonté d'illustrer une démarche, non pour une réexécution (33425800 de 2021).

Lorsque le code est dans le corps du texte, les niveaux d'indentation et d'accolades sont conservés et les erreurs de formatage des caractères spéciaux sont évitées. On peut quand même préférer que le code soit copiable : PDF soit OCRisé, pour permettre le copier-coller dans un interpréteur, comme c'est le cas de 31763386, qui présente une requête GraphQL en entier. Ceci favoriserait aussi le Principe -F- pour les machines.

Code protégé

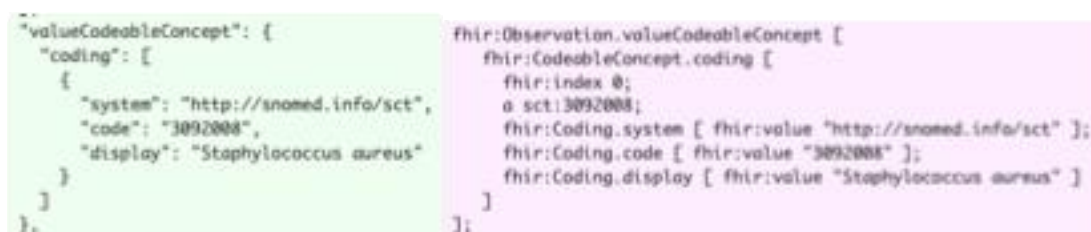
Le cas ci-dessus est particulièrement vrai lorsque le logiciel principal est propriétaire (ce qui n'est évidemment pas FAIR, par exemple 33425800 ci-dessus et son code SAS®).

Pour illustrer un pattern

C'est le cas lorsque le code illustre un standard ou des spécifications formelles. Ainsi, Les auteurs ont jugé qu'il n'était pas utile de faire lire ce code par une machine. Cette démarche concerne les standards et vise justement l'application du Principe -I-. À titre d'exemple, 33936490 indique comment appliquer le standard FHIR RDF (standard de structuration des données en de santé). Dans ce cas, l'information est une structure à saisir visuellement par un humain. Le Principe -F- n'a pas été jugé crucial pour des fragments de code.

Figure 19 : Exemple de mise à disposition de code au format image

For example, the FHIR RDF specification states that a triple supplying a concept IRI should be added to the FHIR Coding element if one can be generated. As another example, the specification also states that JSON list ordering should be made explicit by the addition of `fhir:index` triples in RDF. These examples are illustrated in Figure 1, drawn from a FHIR Observation sample, which shows the JSON representation of a coded concept and the RDF equivalent in Turtle¹².



```
"valueCodeableConcept": {
  "coding": [
    {
      "system": "http://snomed.info/sct",
      "code": "3092008",
      "display": "Staphylococcus aureus"
    }
  ]
},
```

```
fhir:Observation.valueCodeableConcept [
  fhir:CodeableConcept.coding [
    fhir:index 0;
    a sct:3092008;
    fhir:Coding.system [ fhir:value "http://snomed.info/sct" ];
    fhir:Coding.code [ fhir:value "3092008" ];
    fhir:Coding.display [ fhir:value "Staphylococcus aureus" ]
  ]
];
```

Figure 1. The JSON representation of a FHIR CodeableConcept property and the RDF equivalent, in Turtle. The subject of this CodeableConcept property is not shown in these snippets.

(Solbrig et al. 2021, pp. 1142-1143. Article n° 33936490)

Pour illustrer le workflow

Le code montre la méthodologie. Il n'est pas un script prêt à exécuter. 39132453 affiche une page qui montre comment charger des données et des paquets, lancer des programmes et récupérer des résultats dans le logiciel py-tetrad. Dans ce cas, le code est du texte au même titre que le texte de l'article, dans une police différente pour être repérable visuellement (-I- pour humains), mais copiable dans (-I- pour machines).

Au vu de ce qui précède, on peut conclure que l'interopérabilité pour les humains peut aller à l'encontre de celle destinée aux machines. Si la pesée des intérêts est faite, c'est à celui qui transmet le code de décider du Principe FAIR à privilégier.

5.5 Réutilisabilité technique

5.5.1 Qualité de la documentation

En matière de code, les Principes -I- et -R- se recoupent. L'analyse de la qualité de la documentation exposée ici concerne également l'interopérabilité.

Les principes FAIR ne visent pas seulement le long terme, mais aussi le partage du code durant la recherche. S'il est admissible qu'un code meure (*sunsetting* selon Sonabend et al. 2024, p. 5) parce qu'il n'est plus utile, il est dommageable que celui-ci prenne fin parce qu'il n'était pas praticable par la communauté, souffrait de bugs, avait une documentation insuffisante ou au contraire inutilement complexe.

La qualité des métadonnées contenues dans la documentation sert tous les Principes FAIR. Nous les analysons ici les ReadMe (ou autres documents ayant la même fonction) des codes dans la mesure où ils permettent leur réutilisabilité.

Pour rappel, l'analyse des critères FAIR peut être menée de manière plus ou moins approfondie. L'analyse du contenu nécessite une grande attention car elle implique l'examen systématique de plusieurs paramètres qualitatifs et quantitatifs tels que :

- La présence ou non d'un document de type ReadMe (variable binaire relevée sous 4.4.3 ci-dessus)
- Le nombre et le nommage des fichiers de documentation (README, LICENCE, INSTALLATION, INDEX...)
- La répartition des informations -R- dans un ou plusieurs fichiers (Licence, provenance : auteurs et contexte du projet, spécifications techniques pour l'exécution...)
- Le standard utilisé pour sa rédaction, le degré de concision ou de détail
- Le format
- Le contenu : Provenance (auteurs, contexte de création, liens vers l'article, données d'autres dépôts) Informations -I- (version du logiciel, dépendances et versions)
- Information d'installation (-R-)

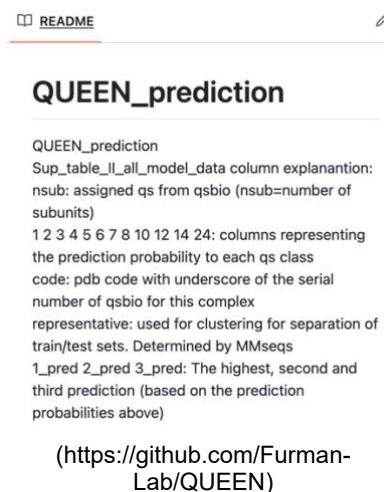
Les pratiques en la matière étant trop irrégulières, les variables ci-dessus ont été examinées systématiquement, mais ni dénombrées ni relevées de manière structurée. (Certains champs sont binaires, d'autres en texte long, les informations sont redondantes dans divers champs selon la pertinence). Nous présenterons plutôt quelques morceaux choisis qui illustrent les bonnes et mauvaises pratiques en matière -R-.

Dans chaque cas, le succès du codem la taille de sa communauté et sa durée de vie sont indiqués dans le but d'identifier les liens éventuels avec les choix des auteurs.

5.5.2 Aspects techniques

Queen (37964216) sur GitHub présente sans introduction l'explication des colonnes du code, en langage naturel, mais très abrégé. Aucun fichier ne précise la licence.

- Vie courte (7 mois entre la création et le dernier commit en septembre 2023)
- Peu de succès (total =1)
- Un seul contributeur, 19 *commits*.

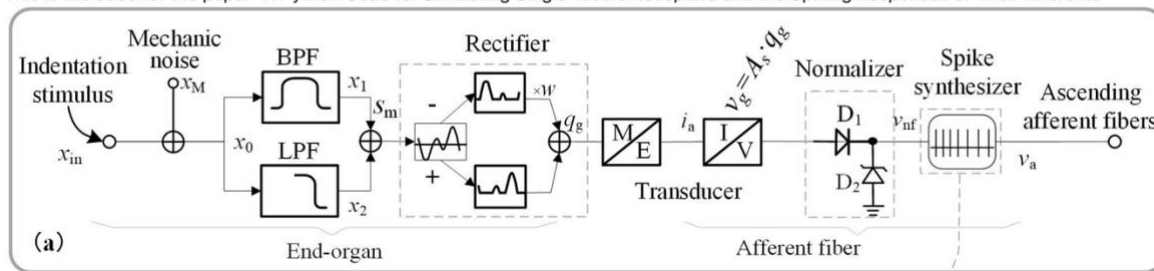


Model_of_single_tactile_unit (39314734) présente le même défaut. Il cite l'article lié par son titre, sans autre référence :

Figure 20 : Exemple de ReadMe insuffisant, purement technique

model_of_single_tactile_unit

This is the code for the paper "A Python Code for Simulating Single Tactile Receptors and the Spiking Responses of Their Afferents"



(https://github.com/ouyangqq/model_of_single_tactile_unit/blob/master/README.md)

- Code ancien et vie courte : janvier à novembre 2019
- Succès très modeste (total =7)
- Un seul contributeur

Figure 21 : Badges des métadonnées sur GitHub

5.5.3 Aspects éthiques

Les auteurs de STAR (23104886) ont créé un code de conduite détaillant les comportements que doit adopter la communauté des contributeurs de ce code : politesse, propos constructifs, prohibition des propos clivants... Celui-ci est calqué sur le modèle de Contributor Covenant.¹² GitHub prévoit en effet un badge pour les codes de conduite.

Par ailleurs, les versions sont indiquées dans des fichiers publiés à plusieurs dates (CHANGES, etc.) qui peuvent porter à confusion.

- Vie assez longue : octobre 2021 à mars 2024
- Grand succès (total = 2713)
- 37 contributeurs, 1210 *commits*

Pour Lemmen et al. (2024, p. 7), ces règles ne sont pas strictement indispensables à l'élaboration d'un code mais elles contribuent grandement à sa qualité.

5.5.4 Règles de contribution

Eeglab (15102499) fournit un document `CONTRIBUTING` qui donne les règles de procédure à suivre pour contribuer au code. Il renvoie à l'onglet *issues* de GitHub qui liste les questions déjà posées pour s'assurer que le bug à reporter en est vraiment un. Le document indique comment nommer la question *issue* pour que celle-ci soit compréhensible et repérable dans l'historique, comment rédiger le message selon les standards de découpage de GitHub,

About

RNA-seq aligner

[Readme](#)

[MIT license](#)

[Code of conduct](#)

[Contributing](#)

[Activity](#)

[2.1k stars](#)

[82 watching](#)

[544 forks](#)

(<https://github.com/alexdobin/STAR>)

¹² A Code of Conduct for Digital Communities. Disponible sur <https://www.contributor-covenant.org/> (consulté le 13 janvier 2026).

comment citer les auteurs de la question, comment qualifier la demande (bug ou amélioration...), etc.

À noter que les auteurs font référence à la documentation dont ils se sont inspirés pour rédiger la leur. Ceci constitue un critère de bonne conduite dans les communautés.

- Vie longue : 9 ans de vie (mars 2014 à nos jours)
- Grand succès (total =1053)
- 39 contributeurs, plus de 13'000 *commits*

5.5.5 Structuration de la documentation

Gromacs (16211538) sur GitHub : README donne la provenance et le contexte, y compris les références bibliographiques, AUTHORS la liste des contributeurs, CITATION.cff les règles de citation, INSTALL-dev l'environnement d'installation (succinctement), CMakeLists.txt les instructions de configuration.

Figure 22 : Arborescence de la documentation sur GitHub

 AUTHORS	LICENCE contient les droits d'utilisation du code, alors que COPYING déclare les droits d'utilisation des œuvres dérivée. Ceux-ci ont été déplacées dans un fichier séparé afin de ne pas gêner la détection automatique de la licence par GitHub et les autres moissonneurs. • Activité incessante depuis décembre 2022 • Grand succès (Total =1231) • 113 contributeurs, plus de 23'000 <i>commits</i>
 CITATION.cff	
 CMakeLists.txt	
 COPYING	
 CPackInit.cmake	
 CTestConfig.cmake	
 INSTALL-dev	
 README	

(<https://github.com/gromacs/gromacs>)

Cette pratique permet d'éviter que les machines ne ratent des informations. À noter qu'il s'agit d'un code à grande communauté et qu'il est déposé sur Zenodo.

La répartition des licences ou d'autres éléments dans plusieurs fichiers peut s'avérer redondante pour un code d'ampleur modeste. Elle augmente les risques d'erreur, même dans un cas de code d'ampleur comme c'est le cas ici, où les informations de citation et de licence figurent dans plusieurs des fichiers. La taille de la communauté influence donc le choix des méthodes d'application des Principes FAIR.

5.5.6 Destinataires de -R-

Les exigences en matière de -I- (pour les machines) et de -R- (pour les machines et les humains) ne sont pas les mêmes. Les auteurs du code suivant ont eu soin de donner la même information de provenance dans deux fichiers séparés :

`README` s'adresse aux humains en langage naturel ou non structuré (-R-) et `DESCRIPTION` est lisible par la plateforme CRAN, par exemple. Ceci permet aux machines de trouver, lire et réutiliser le code (-F-, -I-, -R-).

Figure 23 : Documentation pour machines v. humains

Blaster

Blaster implements an efficient BLAST-like sequence comparison algorithm, fast and dependency-free. The code base of Blaster is available at <https://github.com/tamminnenlab/blaster>.

Installation

From [Conda](#)

```
conda install -c conda-forge r-blaster
```

Development version from GitHub

```
devtools::install_github("tamminnenlab/blaster")
```

Examples

```
# Read a query file into DataFrame
query <- read_fasta("inst/extdata/query.fasta")

# Read a database file into a DataFrame
db <- read_fasta("inst/extdata/db.fasta")

# BLAST the query against the database
blast_table <- blast(query, db)

# BLAST protein sequence file against itself using fileena
```

DESCRIPTION

```
Package: blaster
Type: Package
Title: Native R Implementation of an Efficient BLAST-Like Algorithm
Version: 1.0.6
Date: 2023-05-10
Authors@R: c(person("Manu", "Tamminen",
  email = "mavata@utu.fi",
  role = c("aut", "cre"),
  comment = c(ORCID = "0000-0001-5891-7653")),
  person("Timothy", "Julian",
  email = "tim.julian@eawag.ch",
  role = c("aut"),
```

`README`
<https://github.com/tamminnenlab/blaster/blob/main/README.md>

Les documentations les plus à la pointe des bonnes pratiques dans notre corpus semblent émises pour des codes à grande communauté. Ceci semble logique dans la mesure où une documentation de qualité demande du temps et l'énergie.

Elle exige également une motivation : les codes qui sont entretenus par un seul contributeur semblent utiliser GitHub comme plateforme pour des raisons pratiques pour le codeur. La documentation succincte n'est là que pour mémoire et n'a pas vocation à être utilisée par les pairs.

On notera que les 3 codes hébergés à la fois sur GitHub et déposés sur Zenodo comportent un ReadMe et une déclaration de licence, ce qui était attendu puisqu'ils sont les seuls à respecter les Principes FAIR du développement et du dépôt.

5.5.7 Aspects juridiques

Selon les Principes FAIR4S, le code doit être accompagné d'une licence clairement déclarée. Les bonnes pratiques courantes demandent que la licence soit Open Source, selon la définition de OSI.¹³ Le Principe -R- veut que la licence confère les droits d'utilisation, mais aussi d'adaptation et de redistribution.

¹³ Notamment The FAIR-BioRS Guidelines, v. 2.0.0, disponible sur sa page GitHub <https://github.com/FAIR-BioRS/Guidelines>, consulté le 8 janvier 2026. (Patel et al. 2023, p. 7).

Licences pour objets statiques

Dans notre corpus, un nombre trop important de codes n'ont pas de licence, probablement parce que les auteurs estiment que les droits des articles s'étendent au code associé.

De nombreux codes ont une licence Creative Commons. Or, celles-ci, adaptées aux données statiques et aux articles, ne traitent pas des aspects spécifiques au code que sont la distribution, la gestion des œuvres dérivées qui seraient créées en utilisant le logiciel, l'intégration de parties du code dans d'autres logiciels et de la compatibilité des licences.

La licence CC ND (Creative Commons NoDerivatives) est la plus incompatible avec les Principes FAIR car elle interdit d'utiliser le code dans des œuvres dérivées. Or, la réutilisation d'un code exige par nature des modifications et des intégrations.

Licences pour le code

Les licences dites « permissives » permettent d'utiliser, modifier et partager le code sans restrictions, y compris à des fins commerciales (Benhamou 2024, p. 3). Elles exigent toutefois de citer les auteurs du code. Dans notre corpus, les licences MIT et BSD répondent à ces critères.

Les licences « weak copyleft » sont plus restrictives et exigent que le code modifié et redistribué soit diffusé sous les mêmes conditions que le code original. Ceci garantit que le code demeure Open source. Dans notre corpus, ce sont les licences LGPL.

Les licences « strong copyleft » imposent que tout logiciel dérivé, lié ou combiné avec le code original soit distribué sous la même licence ou une licence compatible, étendant ainsi l'obligation de partage à l'ensemble du logiciel. Dans notre corpus, cette catégorie correspond à la licence GNU GPL.

Les licences strong copyleft ne sont pas considérées comme FAIR, car elles présentent le risque que le code développé ultérieurement ait une licence incompatible avec des licences plus permissives des dépendances. (Sonabend 2024, p. 3)

Or, dans notre corpus, les codes sous licence GPL sont au nombre de 12. Il est malheureusement impossible de déterminer si les auteurs ont choisi une telle licence par méconnaissance des principes de l'Open Source ou pour d'autres raisons.

6. Bilan

6.1.1 Nature du FAIR

Les Principes FAIR ne sont pas des règles standards applicables de manière uniforme et vérifiables de manière structurée.

- Ils ne sont pas binaires (un comportement n'est pas « FAIR ou non FAIR »)
- Ils ne sont pas quantifiables
- Les critères de mise en pratique évoluent avec la technologie
- Leur bonne implémentation est une question de degrés

- Le même principe peut être servi par des mesures contraires et nécessite une pesée des intérêts
- Leur application dépend de la taille de la communauté du code

A la lumière de l'étude exploratoire et de la littérature, on peut affirmer les choses suivantes :

La pratique FAIR actuelle impose d'héberger son code sur GitHub et de le déposer sur Zenodo. Les institutions qui financent la recherche devraient sensibiliser les chercheurs à la nécessité du dépôt avec identifiant pérenne.

L'information aux chercheurs en matière d'Open science ne peut qu'être bénéfique à l'avancement de la recherche. En particulier, les chercheurs qui produisent du code sans être des codeurs professionnels bénéficieraient grandement de formations sur la structuration des métadonnées et sur les enjeux de la licence.

Les chercheurs devraient en particulier être conscients que des formats interopérables et une licence ouverte ne nuisent pas à leur travail mais le mettent en valeur.

Dans tous les cas, les principes vers lesquels tendre ne peuvent pas être établis dans un catalogue de choix FAIR à respecter à 100%. Ils sont des guides que tout chercheur doit avoir à l'esprit lorsqu'il prend des décisions de gestion et de partage des données de sa recherche. (Barker et. al. 2024).

6.1.2 Le bon et le mauvais élève

Pour illustrer nos propos, nous présentons ici un bon et un mauvais élève FAIR.

Bon élève

TamminenLab (Blaster, NSearch et Npysearch : 38032878) est notre bon élève :

Les codes sont mis à disposition sur GitHub (plusieurs dépôts) et Zenodo. Leurs packages sont distribués sur PyPi, CRAN et Anaconda. La visibilité est grande, la redondance aussi.

Les liens vers les codes sont indiqués dans le corps du texte de l'article. D'autres permettent de naviguer entre les différentes plateformes. Le dataset est également gelé sur Zenodo.

Les métadonnées sont riches, l'arborescence de la documentation sur GitHub et Zenodo est bien structurée, le versioning est systématique et visuellement clair dans les dossiers du code. La documentation à l'égard des humains et des machines est différenciée.

L'article a une licence CC-BY 4.0. Le code est sous licence BSD.

Il a une communauté modeste (3 contributeurs et un coefficient succès à 19). Néanmoins, sa durée de vie est longue (2017 à 2023) ce qui indique que les pratiques de ses contributeurs sont efficaces pour le partage. C'est l'objectif des Principes FAIR.

Mauvais élève

NucPred (17332022) contrevient à tous les Principes FAIR :

Le code est disponible en contactant l'auteur uniquement, ce qui ne constitue pas un protocole libre et ouvert. Le code est annoncé dans le corps du texte de l'article mais le lien vers le site

de l'Université de Stockholm est un lien mort. La licence du code est restrictive (GNU GPL). Le code n'étant pas disponible, on ne peut rien conclure de sa qualité.

6.1.3 Proposition d'outils

Le cas de nos « élèves » ci-dessus met en évidence les différences concrètes de pratiques de gestion de code entre les chercheurs.

Pour faciliter l'application concrète des bonnes pratiques que nous avons identifiées dans ce travail, une checklist opérationnelle ainsi qu'un référentiel FAIR détaillés sont proposés en annexe. Ils sont fondés sur la littérature mais font aussi l'objet de choix subjectifs basés sur l'expérience-utilisateur acquise lors de la navigation dans les plateformes et la documentation des logiciels.

Nous espérons que ces outils permettront aux chercheurs et aux gestionnaires de données d'évaluer et améliorer la réutilisabilité, l'interopérabilité et la traçabilité du code associé aux publications scientifiques.

7. Conclusion

Cette recherche tend à montrer que les Principes FAIR en matière de gestion de code comme données de la recherche sont implémentés de manière irrégulière en biomédecine.

Toutefois, l'échantillon étudié porte exclusivement sur ce domaine et ne concerne qu'une partie infime des articles déposés sur PubMed. Il serait nécessaire de croiser les données obtenues avec celles de corpus issus d'autres domaines scientifiques.

Les Principes FAIR ne sont certes pas implémentables selon des critères universels, mais la littérature s'accorde sur un grand nombre de choix FAIR. On rencontre peu de conseils diamétralement opposés. Nous avons donc tenté de formuler des propositions pratiques sous la forme d'une checklist et d'un référentiel FAIR. Les ajustements suite à des recherches dans d'autres domaines de la science sont les bienvenus.

Bibliographie

- ABDILL, Richard et al., 2024. A how-to guide for code sharing in biology. *PLOS Biology*. Vol. 22, no 9. DOI <https://doi.org/10.1371/journal.pbio.3002815>.
- BARKER, Michelle et al., 2022. Introducing the FAIR Principles for research software. *Scientific Data*. Vol. 9 (2022), no 622. DOI 10.1038/s41597-022-01710-x.
- BARKER, Michelle et al., 2024. The FAIR for Research Software Principles after two years: an adoption update. [en ligne]. 20 mars 2024. Disponible à l'adresse : <https://www.researchsoft.org/blog/2024-03/> [consulté le 16 janvier 2026].
- BENHAMOU, Yaniv, 2024. Open Source AI: Does the Copyleft Clause Propagate to Proprietary AI Models? Revisiting the Definition of Derivatives in the AI-context. [pre-print disponible chez SSRN]. DOI <http://dx.doi.org/10.2139/ssrn.4859623>.
- CHUE HONG, Neil P. et al., 2022. FAIR Principles for Research Software version 1.0. (FAIR4RS Principles v1.0). *Research Data Alliance*. DOI 10.15497/RDA00068.
- COURTES, Ludovic et al., 2024. Source Code Archiving to the Rescue of Reproducible Deployment. 2024 ACM Conference on Reproducibility and Replicability, Jun 2024, Rennes, France. DOI 10.1145/3641525.3663622.
- DEEKSHITHA, 2024. RSMM: A Framework to Assess Maturity of Research Software Project. *arXiv*. DOI doi.org/10.48550/arxiv.2406.01788. [en ligne].
- ESCAMILLA, Emily et al., 2023. It's Not Just GitHub: Identifying Data and Software Sources Included in Publications. DOI 10.48550/arxiv.2307.14469.
- GOMES, Dylan G. E. et al., 2022. Why don't we share data and code? Perceived barriers and benefits to public archiving practices. *Proceedings of the Royal Society B: Biological Sciences*. Vol. 289 (2022), no 1987, pp. 1-11. DOI 10.1098/rspb.2022.1113.
- GROSSMANN, Yves Vincent et al., 2024. Software Management Plans – Current Concepts, Tools, and Application. *Data Science Journal*. Vol. 23, no 43, pp. 1-16. DOI 10.5334/dsj-2024-043.
- GRÜNPETER, Morane et al., 2021. Defining Research Software: a controversial discussion (version 1). Summary Report of FAIR4RS Subgroup 3 activity and discussion. *Zenodo*. DOI <https://doi.org/10.5281/zenodo.5504016>.
- LAMPRECHT, Anna-Lena et al., 2020. Towards FAIR principles for research software. *Data Science*. Vol. 3 (2020), pp. 37-59. DOI 10.3233/DS-190026.
- PATEL, Bhavesh et al., 2023. Making Biomedical Research Software FAIR: Actionable Step-by-step Guidelines with a User-support Tool. *Scientific Data*. Vol. 10 (2023), no 557. DOI [org/10.1038/s41597-023-02463-x](https://doi.org/10.1038/s41597-023-02463-x).
- SOLBRIG Harold R. et al. 2021, 2021. Exploring JSON-LD as an Executable Definition of FHIR RDF to Enable Semantics of FHIR Data. *AMIA Annual Symposium Proceedings Archives*. Vol. 2020, pp. 1140-1149.

SONABEND, Raphael et al., 2024. FAIR-USE4OS: Guidelines for creating impactful open-source software. *PLOS Computational Biology*. DOI 10.1371/journal.pcbi.1012045.

STIEGLITZ, Stefan et al., 2020. When are researchers willing to share their data? – Impacts of values and uncertainty on open data in academia. *PLoS ONE*. Vol. 15, no 7. DOI <https://doi.org/10.1371/journal.pone.0234172>.

TRISOVIC, Ana et al., 2022. A large-scale study on research code quality and execution. *Scientific Data*. Vol. 9 (2022), no 60. DOI doi.org/10.1038/s41597-022-01143-6.

VALLET, Nicolas et al., 2022. Toward practical transparent verifiable and long-term reproducible research using Guix. *Scientific Data*. Vol. 9 (2022), no 597. DOI doi.org/10.1038/s41597-022-01720-9.

WILKINSON, Mark D. et al., 2016. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*. Vol. 3 (2016), no 160018. DOI <https://doi.org/10.1038/sdata.2016.18>.

WILKINSON, Mark D. et al., 2017. Interoperability and FAIRness through a novel combination of Web technologies. *PeerJ Computer Science*. Vol. 3, no 110. DOI 10.7717/peerj-cs.110.

Annexe 1 : Référentiel FAIR pour la gestion de code de la recherche

Réalisé par Laetitia Reversy & Jérôme Roth
dir. Prof. Patrick Ruch. Master en Sciences de l'information, Janvier 2026.

Pour chaque critère, des choix de gestion sont décrits. Les choix de la colonne * * * sont les meilleures pratiques, ceux sous * * sont admissibles. Ceux mentionnés sous colonne * ne sont pas suffisants au regard des Principes FAIR.

CRITERES FAIR DE L'ARTICLE SCIENTIFIQUE

	Principes visés	Obligatoire	* * *	* *	*
Licence	R	✓	+ Licence + Permissive (CC-BY...)	+ Licence + avec restrictions	+ Pas de licence + ou droits réservés
DOI	F, R	✓	+ OUI		+ NON
Lien vers le code dans le corps du texte	F	✓	+ Lien vers forge + Lien vers Zenodo + Lien vers autres plateformes	+ Lien vers Zenodo seulement	+ NON

CRITERES FAIR DU CODE

	Principes visés	Obligatoire	***	**	*
SMP (Software Management Plan)	FAIR	Selon ampleur	Réalisé	N'existe pas mais le code et sa communauté sont de taille très modeste	
Hébergement GitHub	FAIR	✓	Réalisé	Hébergement sur une forge ne permettant pas la génération d'un DOI automatique	Pas de plateforme de versionnage
API vers Zenodo (DOI)	F, A, R	✓	API GitHub => Zenodo	Dépôts manuels sur Zenodo ou autre dépôt	Pas de dépôt (pas de DOI)
Packages sur plateforme de distribution	F, R	NON	Réalisé		NON
Push sur Software Heritage	F, R	✓	Réalisé dès le premier release	Push manuel après publication	NON
Licence	R	✓	• Licence permissive • Dans document adhoc	• Licence permissive • Incorporée au ReadMe	• Pas de déclaration de licence • ou lien mort vers la licence
ReadMe	F, R	✓	Selon ampleur : • Un document de référence ('README') qui pointe vers le reste de la documentation • ou un README contenant toutes les informations nécessaires	• README lacunaire • ou mélange des informations pour machines et humains	Pas de document de référence type README
Provenance	R	✓	• Auteurs • Contexte de création (Date, cas d'usage...) • Lien vers article • Version • Lien vers autres plateformes		NON
DOI	F, R	✓	DOI Conceptuel et DOI des versions précédentes	DOI Conceptuel seulement	NON
Install	R (humain)	Selon ampleur	Document expliquant comment installer le code		NON
Compilation	R (humain)	Selon complexité	(Type build.md) Nécessaire uniquement si build complexe. Peut être intégré dans INSTALL.md		NON
Citation.cff	F, R (machine et humain)	✓	Dans un fichier type CITATION.cff	Dans le ReadMe principal	NON
Documentation structurée machine	F, L, R (machine)	✓	OUI		NON
Métadonnées structurées	F, R (machine)	✓	• nom du logiciel • version • auteurs • licence • langage(s) • URL, DOI, identifiants	Incomplètes	NON
Dépendances	R (humain)	✓	Liste des dépendances		NON
Compilation	R (machine)	✓	Instructions de compilation (setup.py / MakeFile)		NON
Image Docker	R (humain)	NON	Renvoyer à une adresse où figure une image docker est un +	NON	

Annexe 2 : Checklist FAIR pour software

Checklist pour la gestion de code source lié à des articles scientifiques selon les principes FAIR

Réalisé par Laetitia Reversy & Jérôme Roth
dir. Prof. Patrick Ruch. Master en Sciences de l'information

- ☐ Dès la création du code, je rédige un Software Management Plan (SMP).
- ☐ Je partage mon code sur GitHub, voire sur d'autres plateformes de versionnage.
- ☐ J'active l'API de GitHub qui génère des DOI sur Zenodo pour chaque version.
- ☐ Je choisis une licence permissive adaptée aux logiciels (MIT ou BSD par exemple. Pas Creative Commons).
- ☐ J'annonce la licence dans un fichier `LICENCE` dédié et l'indique dans le champ prévu à cet effet sur toutes les plateformes où se trouve mon code.
- ☐ Mon code contient un fichier texte `README` indiquant les données de provenance : les contributeurs et le contexte de création, y.c. le DOI de l'article scientifique.
- ☐ Le ReadMe indique le DOI conceptuel du code tel que déposé sur Zenodo ainsi que le lien vers les DOIs de chaque version.
- ☐ Je structure la documentation selon mon SMP et réparties les informations destinées aux machines et aux humains dans des documents distincts.
- ☐ Ma documentation indique, pour chaque dépendance, son rôle dans le projet, son lien officiel, sa version, son langage d'exécution, sa licence, sa procédure d'installation et les remarques sur sa compatibilité.
- ☐ Ma documentation décrit le code avec un langage contrôlé (ontologies, vocabulaires reconnus par ma communauté) afin d'être compréhensible pour les humains.
- ☐ Ma documentation inclut des balises standardisées (ex. schema.org) pour assurer l'interopérabilité et sa découvrabilité par les machines.
- ☐ Dans ma publication, je donne les liens vers GitHub, le dépôt Zenodo et toute autre plateforme où le code est hébergé.
- ☐ Je « pousse » mon code sur Software Heritage.

v.2, janvier 2026

Annexe 3 : Méthodologie détaillée

Également disponible sous `methodologie.md` dans le dépôt Zenodo et sur GitHub

PHASE EXPLORATOIRE

Requêtes de quelques articles depuis PubMed

Equations requêtées sur PubMed (<https://pmc.ncbi.nlm.nih.gov/>)

Language recherché	Requête	Nb articles retrieved	ID article
json	json	2	PMC8075421 et PMC5796402
python	name== python	2	PMC10809901 et PMC8102371

Analyse intuitive des résultats sur `Analyse_explo.xls`: La requête n'étant pas précise, on a trouvé 3 résultats où le code se trouve soit:

- dans le corps même de l'article,
- comme lien dans le texte de l'article,
- comme image (par exemple .pdf) dans le corps de l'article

Cependant, le "code" associé n'en est pas vraiment. Il s'agit de

- renvoi à des vocabulaires. Par exemple FHIR, une norme développée par LH7 pour l'échange de données de santé entre institutions médicales, '<https://hl7.org/fhir/datatypes.html#Coding>') ou '<http://xmlns.com/>', domaine web créé pour nommer des concepts lors de l'utilisation de vocabulaires (RDF, XML, etc.).
- une démonstration de Jupyter Notebook sur la page des suppléments de PubMed pour montrer comment structurer des données, en html.
- de l'illustration de statistiques liées aux données de recherche, dans un format nécessitant la souscription à un logiciel propriétaire (Adobe).

STRUCTURATION DE LA BASE DE DONNEES

- Sur la base des résultats obtenus et de l'estimation des besoins: Élaboration d'une structure pour base de données dans un document 2025_05_03_STRUCTURE_AIRTABLE.rtf
- Intégration dans [Airtable.com](https://airtable.com). Adaptation des champs et tables aux besoins et création de formulaires de requêtes :
 - Formulaire 'Nouvel article' : insère chaque article retrieved dans la table ARTICLES
 - Formulaire 'Associer Code' : Pour chaque article, introduction d'une ou plusieurs formes de mise à disposition (dépôt sur plateforme d'archivage, sur plateforme de développement, etc.) dans la table MISEaDISPO (une ligne par mise à disposition : contrairement à un tableur, la base de données permet d'assigner plusieurs mises à disposition au même code).

Rapidement, les formulaires se sont avérés désuets. L'adaptation de la base aux résultats a nécessité de travailler directement en grid view, ce qui convient du fait de la taille modeste du corpus et qui permet d'évaluer les critères subjectifs de visu, de manière empirique, afin d'affiner les critères.

METHODOLOGIE POUR LE RETRIEVAL DE CODE

Choix du code

La méthode de requête est fondée sur un fichier 'suppfiles_code.csv' extrait de PubMed par M. Gobeill le 25/02/2025 et qui renvoie 13'000 articles scientifiques contenant des fichiers de code. Il compte 13'000 lignes avec un pmcid, l'extension du fichier, et son nom. Ont été exclus les fichiers dont l'extension ne correspondait pas à un fichier informatique, sur la base d'une liste fournie par M. Ruch, 'extension.json'. [PAS COMPRIS A QUOI ELLE SERVAIT] : M. Gobeill a exclu de la requête les extensions "txt", "csv", "html", "xml", "eps", et "htm", qui ne sont pas vraiment du code.

Notre intention première est de trouver du code dans les 11 langages qui ressortent le plus souvent du fichier 'suppfiles_code.csv', à savoir:

'R' 'm' 'mat' 'py' 'pl' 'sps' 'sas' 'c' 'sh' 'cpp' 'java'

(A noter que la liste ci-dessus sera adaptée par la suite, voir ci-dessous). Nous avons utilisé ChatGPT pour suggérer des requêtes.

Requêtes formulées:

Base interrogée : <https://pubmed.ncbi.nlm.nih.gov/>. Un seul article (38237907) a été recherché sur <https://pmc.ncbi.nlm.nih.gov/>, par inadvertance.

Langage recherché	Requêtes	Nb articles retrieved
C++ ou Cpp	("C++" OR "C plus plus") AND (code OR source code OR implementation OR software OR algorithm)	5
py	("python" OR "py") AND code	3
py *	(NCBI C++ Toolkit)	2
py	(name == python)	2
mat ou matlab	(matlab OR ".mat" OR "MAT-file" OR "mat-file") AND (code OR script OR software OR "source code")	5
java	java code	5
json **	json	2

* Requête pour trouver du C++ mais a permis de trouver du python.

** Pas du langage procédural mais ces requêtes ont été faites lors de la phase test de la base de données. Ensuite, on s'est concentré sur des langages procéduraux.

Remarque concernant les droits d'accès:

- les articles qui ne sont pas en accès libre (paiement requis ou accès via une institution) ont été écartés
- En revanche, les articles libre d'accès contenant du code dont l'accès est restreint ont été retenus, ceci afin de mettre en évidence les pratiques non recommandées de diffusion de code.

Ecueils dans le choix des requêtes et langages

Orthographes différentes pour un même langage

Certaines des 11 extensions choisies sont difficiles à chercher séparément.

- **Extensions ressemblantes** : Il est difficile de formuler des équations ciblées pour mat et matlab. En général, les résultats sont congruents. On a donc choisi de grouper les recherches .mat et .matlab.

- **Mêmes langages avec différentes extensions:** les fichiers C++ ont souvent comme extension cpp, pour éviter les erreurs de lecture. On a donc groupé ces deux recherches en une.

Nous avons donc décidé d'ajouter à notre liste de 11 extensions autant d'éléments que nécessaires pour obtenir 55 résultats. (voir la liste totale des requêtes plus bas dans la procédure).

Notion de code

Le Prof. Gobeill a exclu de son fichier les langages de code de structuration ou de représentation pour ne garder que les codes procéduraux / exécutifs. Ceci peut porter un biais sur la recherche. Toutefois, lors d'une première recherche de code en .json, on s'est rendu compte que ce type de code n'était pas mis à disposition selon les règles FAIR, c'est à dire qu'il n'était pas uploadé sur des sites externes à PubMed.

Dans la table PLATEFORME, qui est un champ lié de la table MaDISPO_CODE, nous avons prévu un champ LocalPubMed. Il vise les cas où le code est disponible dans le corps même de l'article ou sur la page PubMed dans les suppléments.

La plupart des codes qui présentaient cette particularité sans être publiés ailleurs étaient sous forme de fichier image (p.x. pdf), parfois en annexe, parfois dans le corps même du texte, souvent non OCR-isés et donc pas exportables dans un éditeur de code. Cet élément, si le code n'est pas publié en externe, peut affecter les paramètres F, I et R.

Les quelques articles renvoyaient donc à des ontologies, des sites de gestion de concepts lors de utilisation de vocabulaires XML ou RDF, du langage html, ou des morceaux de code tels qu'ils devaient apparaître lorsque l'on exécute un logiciel par ailleurs déjà connu des lecteurs de l'article.

Le code de structuration/représentation ne nécessite pas d'être déployé de manière détaillée à l'usage du lecteur de l'article: le code en lui-même existe déjà et ne doit pas être réinventé par les auteurs. L'exclusion des langages tels que .json non procéduraux dans la liste des codes nous a donc paru pertinente et nous avons décidé de poursuivre notre recherche sur la base de la liste de M. Gobeill.

Exemples de requête json:

Requête	Langage trouvé	Forme de Mise à dispo
json	json	Lien vers .pdf sur un site d'ontologie (FHIR)
	xml	Screenshot sur Page des Suppléments
	owl (langage descriptif)	Lien dans le texte vers ontologie
	html	Jupyter notebook

Requêtes suivantes : Comme indiqué plus haut [NUMEROTER PRECISEMENT LE RENVOI...], nous avons ajouté à notre liste trois extensions:

- ipynb
- sps
- NetLogo

Langage recherché	Requête	Nb articles retrieved
R	r-script	3
R	r-code	2
perl	perl-code	5
sas	sas code	5
shell	"bash script"[tiab]	5
ipynb	ipynb	5
sps	SPSS script	4 *
NetLogo	netlogo	4

* Nous n'avons à ce jour pas trouvé de 5ème article contenant du code sps. Les articles en SSPS sont anciens (articles trouvés jusqu'en 2004, soit avant GitHub) et les habitudes de publication de code n'étaient pas encore ancrées. Nous avons donc ajouté NetLogo, qui est la suivante dans la liste des extensions les plus utilisées et qui soit du langage procédural.

ypinb n'est pas un langage procuédural mais un conteneur. Toutefois, les autres extensions de langage procédurales plus fréquentes (.c et .do) sont très difficiles à requêter dans PubMed, d'où le choix de l'extension .ipynb.

AMENDEMENTS DE LA BASE DE DONNEES

Au cours de l'encodage des métadonnées dans la base, des résultats sont apparus de visu et de manière empirique, ce qui a suscité un affinage de la base. Les champs suivants ont été ajoutés:

Champs non exploités

Car jugés non pertinents lors de l'analyse exploratoire

- Domaine
- Pays
- Langues

Champs ajoutés

Succès

Ajout de 4 champs concernant le succès du code:

- Like/stars/supporters
- Vues
- Forks
- Pull requests

Ces champs n'étant pas tous disponible sur les plateformes différentes, ajout d'un champ en fin de processus: TotalSuccesCode qui additionne les 4 colonnes ci-dessus.

Licence

Ajout d'un champ RegroupLicences pour déterminer, pour chaque licence, si elle est adaptée à l'objet code: 5 entrées:

- Licence prévue pour le code (i.e. adaptée)
- CC (créative common) ou autre licence non adaptée
- Reserved (droits restreints)
- Licence pas déclarée (absence de licence)
- N/A car licence pas accessible

CONVERSION DES ARTICLES EN HTML

Méthode Conversion des articles de .pdf au format html à l'aide de l'outil open source pdftohtml (suite Poppler, version installée via Homebrew 23.1.0), exécuté en ligne de commande dans le Terminal macOS: `pdftohtml -c -s -enc UTF-8 *.pdf`

Articles non convertis :

- 11238077.pdf protégé par copyright
- 32096823.pdf idem

Problèmes de formatage:

- 33425800.pdf (Syntax Warning: Invalid Font Weight)

Abandon de la démarche.

EXPLOITATION DES DONNEES ET EXPORTS

Choix de la plateforme

En prévision de la présentation de la recherche par poster (fin décembre 2025), la statistique suivante a été effectuée: choix de la plateforme: Zenodo 10%, GitHub 55%, Autres plateformes de développement 2%, plateformes de distribution de packages 6% et pas ou peu de gestion du code 27%. Pour ce faire, un export de la table MaDISPO_CODE a été fait et retravaillé (voir 2025 11 24 MaDISPO_CODE-Grid view.csv).

Les virgules dans les exports provoquent parfois des décalages lors de la conversion .csv en .xls avec tableur. Corrections à l'œil.

A noter que la table a été modifiée après publication du poster, une entrée de la table MaDISPO (PMID 38032878) a été retirée (8 au lieu de 9 entrées) car le dépôt sur Zenodo ne concernait que les données.

Nombre d'articles

Corrections avant exploitation:

- Article 36951911 supprimé. Pas de code lié. Restent 58 articles, 81 mises à disposition.
- NB, les articles 24268083 et 29317802 sont indisponibles bien qu'annoncés. 17332022 et 24197709 ne sont que disponibles en contactant l'auteur. Ils ont été conservés car montrent une mauvaise pratique. Nombre de mises à disposition pris en compte pour la rédaction du billet de blog:

58 articles

Une entrée de la table Articles s'est avérée vide. nouveau nombre pris en compte:

57 articles

Choix de la mise à disposition

Type de requêtes en langage naturel dans Omni (IA de AirTable) : peux-tu faire une liste basée sur Table articles, champ PMID et sur table MaDISPO_CODE champ Plateforme, qui dit X articles a mis à disposition sur Plateforme 1, X sur palteforme 2, etc. et compléter la liste avec les recoupements (parmi eux, X ont mis a disposiiton à la fois sur plateforme 1 et plateforme 2).

Vérification systématique de la cohérence des réponses avec les résultats déjà obtenus, en effectuant des checks dans la base de données et en posant des questions croisées. Omni se trompe dans ses décomptes.

Findability

Emplacement du lien du code: Pour déterminer si les liens étaient dans le corps de l'article, ailleurs, ou inexistant, des exports ont été faits en csv, transformés en xls et classés. La vérification à la main s'imposait, du fait de l'encodage en langage naturel et différents champs concernés par le même paramètre.

Même méthode appliquée aux critères suivants:

- **Présence d'un ReadMe**
- **Choix de la licence**

VISUALISATIONS

Les visualisations ont été faites sur la base des exports .csv de la base de données ou des conversions en.xls, soit par l'outil de visualisation de Excel, soit par Power BI.